

RTL Design Sherpa

RTL AMBA AXI5-Stream Module Reference — Rev 0.92

October 2026

Table of Contents

1 AXIS5 (AXI5-Stream) Modules....10
1.1 Overview....11
1.2 Implemented Signal Set....12
1.3 AXIS4 vs AXIS5 in This Library....15
1.4 Module Categories....17
1.4.1 Stream Master Components....17
1.4.2 Stream Slave Components....17
1.5 Key Features....18
1.5.1 AXI5-Stream Protocol Support....18
1.5.2 AXI5 and Extension Features....18
1.5.3 Power Management....18
1.5.4 Performance....18
1.6 Quick Start....19
1.6.1 Using AXI5-Stream Master....19
1.6.2 Using AXI5-Stream Slave....20
1.6.3 Clock-Gated Streaming....21
1.7 Testing....23
1.8 Protocol Details....24
1.8.1 AXI5-Stream Signal Descriptions....24
1.8.2 Flow Control....24
1.8.3 Byte Qualification....25
1.9 Design Notes....26

1.9.1 FUB Interface Pattern....26
1.9.2 Migration from AXI4-Stream....26
1.9.3 Streaming Pipeline Integration....26
1.10 Performance Characteristics....28
1.11 Related Documentation....29
1.12 References....30
1.12.1 Specifications....30
1.12.2 Papers and textbooks....30
1.12.3 Source Code....31
1.13 Navigation....32
2 AXIS5 Master....33
2.1 Overview....34
2.1.1 Key Features....34
2.1.2 AXIS5 Extensions Over AXIS4....34
2.2 Parameters....36
2.2.1 Derived Values (do not override)....36
2.3 Ports....38
2.3.1 Clock and Reset....38
2.3.2 FUB AXIS5 Interface (Input Side)....38
2.3.3 Master AXIS5 Interface (Output Side)....39
2.3.4 Status Outputs....40
2.4 Functional Description....41
2.4.1 Block diagram....41
2.4.2 Skid Buffer Operation....41

2.4.3 Packet Packing/Unpacking....42
2.4.4 Interface Parity (ENABLE_PARITY=1)....42
2.4.5 Busy Signal....44
2.5 Timing Characteristics....45
2.5.1 Basic Transfer with Wake-up....45
2.5.2 Transfer with Parity Error....45
2.5.3 Skid Buffer Backpressure....45
2.6 Usage Examples....46
2.6.1 Basic Configuration....46
2.6.2 With Parity Protection....47
2.7 Design Notes....49
2.7.1 AXIS5 vs AXIS4 Differences....49
2.7.2 Skid Buffer Sizing....49
2.7.3 Parity Implementation....49
2.7.4 Optional Signal Widths....50
2.8 Related Modules....51
2.9 Testing....52
2.10 References....53
2.11 Navigation....54
3 AXIS5 Master with Clock Gating....55
3.1 Overview....56
3.1.1 Key Features....56
3.1.2 Power Management Features....56
3.2 Parameters....57

3.2.1 Derived Values (do not override)....57
3.3 Ports....59
3.3.1 Clock and Reset....59
3.3.2 Clock Gating Configuration....59
3.3.3 FUB AXIS5 Interface (Input Side)....59
3.3.4 Master AXIS5 Interface (Output Side)....60
3.3.5 Status Outputs....61
3.4 Functional Description....63
3.4.1 Block diagram....63
3.4.2 Clock Gating Control....63
3.4.3 Wake-up and Gating Latency....63
3.4.4 Reset Behavior....64
3.4.5 Power Saving Mechanism....65
3.4.6 Integration with AXIS5 Extensions....65
3.4.7 Idle Count Configuration....65
3.5 Timing Characteristics....66
3.5.1 Clock Gating Activation....66
3.5.2 Wake-up Signal Preventing Gating....66
3.5.3 Transfer During Idle Period....66
3.6 Usage Examples....67
3.6.1 Basic Clock-Gated Configuration....67
3.6.2 Dynamic Clock Gating Control....68
3.6.3 Power Profiling Example....69
3.7 Design Notes....71

3.7.1 Clock Gating vs. Non-Gated Variant....71
3.7.2 Wake-up Integration Benefits....71
3.7.3 Gate Transition Overhead....71
3.7.4 Clock Domain Considerations....71
3.8 Related Modules....73
3.9 Testing....74
3.10 References....75
3.11 Navigation....76
4 AXIS5 Slave....77
4.1 Overview....78
4.1.1 Key Features....78
4.1.2 AXIS5 Extensions Over AXIS4....78
4.2 Parameters....80
4.2.1 Derived Values (do not override)....80
4.3 Ports....82
4.3.1 Clock and Reset....82
4.3.2 Slave AXIS5 Interface (Input Side)....82
4.3.3 FUB AXIS5 Interface (Output Side to Backend)....83
4.3.4 Status Outputs....84
4.4 Functional Description....85
4.4.1 Block diagram....85
4.4.2 Skid Buffer Operation....85
4.4.3 Packet Packing/Unpacking....86
4.4.4 Interface Parity (ENABLE_PARITY=1)....86

4.4.5 Busy Signal....87
4.5 Timing Characteristics....88
4.5.1 Basic Receive with Wake-up....88
4.5.2 Receive with Parity Error....88
4.5.3 Skid Buffer Backpressure from Backend....88
4.6 Usage Examples....89
4.7 Design Notes....91
4.7.1 AXIS5 vs AXIS4 Differences....91
4.7.2 Parity Check Location....91
4.7.3 Skid Buffer Sizing....91
4.7.4 Parity Implementation....91
4.7.5 Error Handling Strategy....92
4.8 Related Modules....93
4.9 Testing....94
4.10 References....95
4.11 Navigation....96
5 AXIS5 Slave with Clock Gating....97
5.1 Overview....98
5.1.1 Key Features....98
5.1.2 Power Management Features....98
5.2 Parameters....99
5.2.1 Derived Values (do not override)....99
5.3 Ports....101
5.3.1 Clock and Reset....101

5.3.2 Clock Gating Configuration....101
5.3.3 Slave AXIS5 Interface (Input Side)....101
5.3.4 FUB AXIS5 Interface (Output Side to Backend)....102
5.3.5 Status Outputs....103
5.4 Functional Description....105
5.4.1 Block diagram....105
5.4.2 Clock Gating Control....105
5.4.3 Wake-up and Gating Latency....105
5.4.4 Reset Behavior....106
5.4.5 Power Saving Mechanism....107
5.4.6 Check-signal Error Detection....107
5.4.7 Idle Count Configuration....107
5.5 Timing Characteristics....108
5.5.1 Clock Gating Activation on Slave....108
5.5.2 Wake-up Preventing Gating During Receive....108
5.5.3 Backend Backpressure with Clock Gating....108
5.6 Usage Examples....110
5.6.1 Basic Clock-Gated Slave....110
5.6.2 With Parity Protection and Power Monitoring....111
5.6.3 Receive Pipeline with Power Management....113
5.7 Design Notes....115
5.7.1 Clock Gating vs. Non-Gated Variant....115
5.7.2 Slave-Specific Gating Considerations....115
5.7.3 Gate Transition Overhead....115

5.7.4 Parity Error Handling with Clock Gating....115
5.7.5 Clock Domain Considerations....116
5.8 Related Modules....117
5.9 Testing....118
5.10 References....119
5.11 Navigation....120

1 AXIS5 (AXI5-Stream) Modules

Location: `rtl/amba/axis5/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The AXIS5 subsystem gives you AXI5-Stream master and slave endpoints, plus clock-gated variants of each, for high-throughput streaming data applications.

AXI5-Stream extends AXI4-Stream primarily with wake-up signalling (TWAKEUP) for low-power operation. These modules implement the AXI4-Stream handshake and signal set, add TWAKEUP on top, and offer the optional interface parity protection of IHI 0051B ch. 5: the *CHK check signals, odd parity, one check bit per 8 bits.

1.2 Implemented Signal Set

Read this table before you integrate with third-party AXI-Stream IP. It's the authoritative statement of what the RTL in `rtl/amba/axis5/` actually carries — not what the ARM spec says could be there.

Signal	Status in these modules	Notes
TDATA	Implemented	Width set by `AXIS_DATA_WIDTH`
TSTRB	Implemented	Width `AXIS_DATA_WIDTH/8`
TKEEP	**Not implemented**	No TKEEP port exists. See "Byte Qualification" below
TLAST	Implemented	Packet/frame boundary
TID	Implemented	Width `AXIS_ID_WIDTH` (default 8, set 0 to remove)
TDEST	Implemented	Width `AXIS_DEST_WIDTH` (default 4, set 0 to remove)
TUSER	Implemented	Width `AXIS_USER_WIDTH` (default 1, set 0 to remove)

Signal	Status in these modules	Notes
TVALID / TREADY	Implemented	Standard handshake
TWAKEUP	Implemented, optional	Gated by `ENABLE_WAKEUP` (default 1)
TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK	Implemented, optional	IHI 0051B ch. 5 check signals (Check_Type = Odd_Parity_Byte_All), gated by `ENABLE_PARITY` (default 0). TKEEPCHK is absent because TKEEP is

Deviations from the ARM AXI-Stream signal set:

- **No TKEEP.** These modules carry TSTRB only. Null bytes (the TKEEP=0 encoding) cannot be expressed. A stream that needs null-byte signalling has to carry that information in TUSER or use a different endpoint.
- **Interface parity follows the spec.** Odd parity (§5.3), one check bit per 8 bits with the top bit covering any short remainder, Table 5-1 names and widths. Until issue #102 these modules carried a single even-parity `TPARITY` port over TDATA only; that port is gone.
- **No TPOISON and no chunking (TCHUNKEN) support.** Neither signal is present in the RTL.
- **TID default width is 8 bits.** `AXIS_ID_WIDTH` is a free parameter, so wider IDs are configurable, but nothing in these modules requires or defaults to a wider ID.

1.3 AXIS4 vs AXIS5 in This Library

This table compares the two generations *as implemented here*, not the full ARM specifications.

Feature	AXIS4 modules (`rtl/amba/axis4/`)	AXIS5 modules (`rtl/amba/axis5/`)
Basic protocol	TVALID/TREADY handshake	TVALID/TREADY handshake
Wake-up	Not present	TWAKEUP, gated by `ENABLE_WAKEUP`
Interface parity	Not present	IHI 0051B ch. 5 `*CHK` check signals plus sticky `parity_error`, gated by `ENABLE_PARITY`
Side-band signals	TID, TDEST, TUSER	TID, TDEST, TUSER (same, parameterized widths)
Byte qualification	TSTRB only	TSTRB only
Clock gating	Available (`_cg` variants)	Available (`_cg` variants)
Clock-gate port names	`cfg_cg_enable` / `cfg_cg_idle_count`	**`cfg_cg_enable`** / **`cfg_cg_idle_count`** – see below
Stream port prefix on the `_cg` wrappers	`fub_axis_*` / `m_axis_*`	**`fub_axis5_*`** / **`m_axis5_*`** – see below

Two naming outliers, both in the axis5 `_cg` wrappers. Learn them before you port an instantiation across, because they will trip you:

- Thirty-two of the thirty-four `_cg` modules in `rtl/amba` take `cfg_cg_enable`. The two exceptions are `axis5_master_cg` and `axis5_slave_cg`, which take `cfg_cg_enable` / `cfg_cg_idle_count`.
- Those same two wrappers carry a `5` in their stream port names (`fub_axis5_tdata`, `m_axis5_tvalid`, …) while the modules they WRAP – `axis5_master`, `axis5_slave` – use `fub_axis_*` / `m_axis_*` / `s_axis_*` without it.

Neither is a defect. Both are inconsistencies that break copy-paste between the axis4 and axis5 pages and between a wrapper and the module inside it. Renaming them is a family-wide interface change and has not been made — so the documentation calls it out instead.

1.4 Module Categories

1.4.1 Stream Master Components

Module	Description	Documentation	Status
axis5_master	AXI5-Stream master for high-throughput data streaming	axis5_master.md	Documented
axis5_master_cg	Clock-gated AXI5-Stream master	axis5_master_cg.md	Documented

1.4.2 Stream Slave Components

Module	Description	Documentation	Status
axis5_slave	AXI5-Stream slave for data reception	axis5_slave.md	Documented
axis5_slave_cg	Clock-gated AXI5-Stream slave	axis5_slave_cg.md	Documented

1.5 Key Features

1.5.1 AXI5-Stream Protocol Support

- **Flow Control:** TVALID/TREADY handshaking with backpressure
- **Packet Boundaries:** TLAST for packet/frame demarcation
- **Byte Qualification:** TSTRB (TKEEP is not implemented - see “Implemented Signal Set”)
- **Side-band Data:** TID, TDEST, TUSER for routing and metadata

1.5.2 AXI5 and Extension Features

- **Wake-up Signaling:** TWAKEUP for low-power state exit
- **Optional Interface Parity:** the IHI 0051B ch. 5 check signals (odd parity, one bit per 8 bits), computed with `rtl/common/dataint_parity`, with a sticky `parity_error` flag
- **Parameterized Side-band Widths:** TID, TDEST, and TUSER widths are set per instance

1.5.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Wake-up Support:** TWAKEUP integration for power management
- **Idle Detection:** Automatic clock gate when stream is idle

1.5.4 Performance

- **Skid-Buffered Path:** All four modules pass data through a `gaxi_skid_buffer`; there is no combinational bypass mode
- **Full-Rate Streaming:** One transfer per clock sustained when downstream keeps TREADY asserted
- **Configurable Widths:** Flexible data and signal widths

1.6 Quick Start

1.6.1 Using AXI5-Stream Master

```
axis5_master #(
    .SKID_DEPTH         (4),
    .AXIS_DATA_WIDTH    (128),
    .AXIS_ID_WIDTH      (8),
    .AXIS_DEST_WIDTH    (4),
    .AXIS_USER_WIDTH    (8),
    .ENABLE_WAKEUP      (1),
    .ENABLE_PARITY      (0)
) u_axis5_master (
    .aclk               (clk),
    .aresetn            (resetn),

    // FUB streaming interface (from internal logic)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tdatachk  ('0),            // check signals: unused when
ENABLE_PARITY=0
    .fub_axis_tstrbchk  ('0),
    .fub_axis_tlastchk  ('0),
    .fub_axis_tidchk    ('0),
    .fub_axis_tdestchk  ('0),
    .fub_axis_tuserchk  ('0),

    // AXI5-Stream master interface
    .m_axis_tdata       (m_tdata),
    .m_axis_tstrb       (m_tstrb),
    .m_axis_tlast       (m_tlast),
    .m_axis_tid         (m_tid),
    .m_axis_tdest       (m_tdest),
    .m_axis_tuser       (m_tuser),
    .m_axis_tvalid      (m_tvalid),
    .m_axis_tready      (m_tready),
    .m_axis_twakeup     (m_twakeup),
    .m_axis_tdatachk    (),            // check signals: unused when
ENABLE_PARITY=0
    .m_axis_tstrbchk    (),
    .m_axis_tlastchk    (),
```

```
    .m_axis_tidchk      (),
    .m_axis_tdestchk    (),
    .m_axis_tuserchk    (),
    .m_axis_tvalidchk   (),
    .m_axis_treadychk   (1'b0),
    .m_axis_twakeupchk  (),

    // Status
    .busy               (m_busy),
    .parity_error       ()              // Unused when
ENABLE_PARITY=0
);
```

1.6.2 Using AXI5-Stream Slave

```
axis5_slave #(
    .SKID_DEPTH         (4),
    .AXIS_DATA_WIDTH    (128),
    .AXIS_ID_WIDTH      (8),
    .AXIS_DEST_WIDTH    (4),
    .AXIS_USER_WIDTH    (8),
    .ENABLE_WAKEUP      (1),
    .ENABLE_PARITY      (0)
) u_axis5_slave (
    .aclk               (clk),
    .aresetn            (resetn),

    // AXI5-Stream slave interface
    .s_axis_tdata       (s_tdata),
    .s_axis_tstrb       (s_tstrb),
    .s_axis_tlast       (s_tlast),
    .s_axis_tid         (s_tid),
    .s_axis_tdest       (s_tdest),
    .s_axis_tuser       (s_tuser),
    .s_axis_tvalid      (s_tvalid),
    .s_axis_tready      (s_tready),
    .s_axis_twakeup     (s_twakeup),
    .s_axis_tdatachk    ('0),           // check signals: unused when
ENABLE_PARITY=0
    .s_axis_tstrbchk    ('0),
    .s_axis_tlastchk    ('0),
    .s_axis_tidchk      ('0),
    .s_axis_tdestchk    ('0),
    .s_axis_tuserchk    ('0),
    .s_axis_tvalidchk   (1'b0),
    .s_axis_treadychk   (),
    .s_axis_twakeupchk  (1'b0),
```

```
    // FUB streaming interface (to internal logic)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tdatachk  (),            // check signals: unused when
ENABLE_PARITY=0
    .fub_axis_tstrbchk  (),
    .fub_axis_tlastchk  (),
    .fub_axis_tidchk    (),
    .fub_axis_tdestchk  (),
    .fub_axis_tuserchk  (),

    // Status
    .busy               (s_busy),
    .parity_error       ()               // Unused when
ENABLE_PARITY=0
);
```

1.6.3 Clock-Gated Streaming

```
// Clock-gated master for power efficiency
axis5_master_cg #(
    .AXIS_DATA_WIDTH        (128),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (clk),
    .aresetn                (resetn),

    // Clock gating control
    .cfg_cg_enable            (1'b1),      // 1 = allow gating, 0 =
clock always on
    .cfg_cg_idle_count        (4'd8),      // Idle cycles before the
clock gates

    // Streaming interfaces (note the axis5_ prefix on the _cg
variants)
    .fub_axis5_tdata        (fub_tdata),
    // ... remaining fub_axis5_* ports ...
    .m_axis5_tdata          (m_tdata),
    // ... remaining m_axis5_* ports ...

    // Status
```

```
    .busy                   (m_busy),
    .parity_error           (),
    .cg_gating      (m_clk_gated)
);
```

Port prefix warning: the clock-gated variants name their streaming ports `fub_axis5_*` and `m_axis5_*` / `s_axis_*`, while the non-gated variants use `fub_axis_*` and `m_axis_*` / `s_axis_*`. Swapping a module for its `_cg` counterpart therefore means renaming every FUB-side and master-side connection — the exact kind of mechanical edit that's easy to get half-done. See each module page for the exact port list.

1.7 Testing

All AXIS5 modules are verified with CocoTB-based testbenches in `val/amba/`:

```
# Run all AXIS5 tests
pytest val/amba/test_axis5*.py -v

# Run specific module tests
pytest val/amba/test_axis5_master.py -v
pytest val/amba/test_axis5_slave.py -v
```

1.8 Protocol Details

1.8.1 AXI5-Stream Signal Descriptions

Signal	Direction	Description
ACLK	Input	Stream clock (`aclk`)
ARESETN	Input	Active-low asynchronous reset (`aresetn`)
TDATA	Master to Slave	Stream data payload
TSTRB	Master to Slave	Byte strobes
TLAST	Master to Slave	End of packet/frame indicator
TID	Master to Slave	Stream identifier
TDEST	Master to Slave	Destination routing information
TUSER	Master to Slave	User-defined sideband data
TWAKEUP	Master to Slave	Wake-up signal (AXI5)
TDATACHK … TUSERCHK, TVALIDCHK, TWAKEUPCHK	Master to Slave	IHI 0051B ch. 5 check signals (optional, `ENABLE_PARITY`)
TREADYCHK	Slave to Master	Check signal for TREADY (optional, `ENABLE_PARITY`)
TVALID	Master to Slave	Data valid indicator
TREADY	Slave to Master	Ready to accept data

TKEEP is deliberately absent from this table because no AXIS5 module implements it.

1.8.2 Flow Control

AXI5-Stream flow control is a plain valid/ready handshake:

1. **Data Transfer:** Occurs when TVALID and TREADY are both high
2. **Backpressure:** Slave deasserts TREADY to pause stream
3. **Source Stall:** Master deasserts TVALID when no data available
4. **Packet End:** TLAST indicates final beat of packet/frame

1.8.3 Byte Qualification

These modules carry TSTRB only. The full ARM encoding, shown here for reference, needs both signals:

TKEEP	TSTRB	Meaning
1	1	Data byte
1	0	Position byte (placeholder)
0	0	Null byte (not transmitted)
0	1	Reserved

What these modules support: only the TKEEP=1 rows are expressible. Treat TSTRB=1 as a data byte and TSTRB=0 as a position byte. Null bytes cannot be signalled. When connecting to IP that drives TKEEP, tie its TKEEP to all-ones and pass byte validity through TSTRB, or place a converter at the boundary.

1.9 Design Notes

1.9.1 FUB Interface Pattern

FUB stands for Functional Unit Block: the design-internal logic on the far side of the endpoint from the external stream bus. All AXIS5 modules use this pattern: - **fub_axis_*** (or **fub_axis5_*** on the _cg variants) signals connect to internal logic - **m_axis_*** or **s_axis_*** signals connect to the external stream bus - A skid buffer sits between the FUB and external interfaces for timing closure. A skid buffer is a small registered elastic buffer (`gaxi_skid_buffer`, depth `SKID_DEPTH`) that absorbs backpressure so TREADY does not have to propagate combinationally across the endpoint.

1.9.2 Migration from AXI4-Stream

AXIS5 modules keep the AXI4-Stream data path intact: - Core protocol unchanged (TVALID/TREADY handshake) - TWAKEUP can be tied to 0 for always-awake operation, or removed entirely with `ENABLE_WAKEUP=0` - The check signals are off by default (`ENABLE_PARITY=0`): the `*chk` ports remain but outputs are tied 0 and inputs ignored - TUSER width can match AXI4-Stream configurations

Neither generation implements TKEEP, so a design already using the AXIS4 modules will not gain or lose byte-qualification capability by moving to AXIS5.

1.9.3 Streaming Pipeline Integration

AXIS5 modules drop cleanly into streaming pipelines:

```
// Streaming data processing pipeline
axis5_master u_input (
    .m_axis_*(stage1_axis)
);

// Processing stage
processing_block u_process (
    .s_axis_*(stage1_axis),
    .m_axis_*(stage2_axis)
);

// Clock domain crossing
gaxi_fifo_async u_cdc (
    .s_*(stage2_axis),
    .m_*(stage3_axis)
);

axis5_slave u_output (
    .s_axis_*(stage3_axis)
);
```

1.10 Performance Characteristics

Treat the figures below as design targets, not measured silicon or post-route results. No synthesis or timing run for these modules is published in this repository, and achievable frequency depends heavily on data width, technology, and the surrounding logic. Order-of-magnitude guidance only — quote them in a design review at your own risk.

Metric	Value	Basis
Throughput	1 transfer per clock	Structural - skid buffer sustains full rate while TREADY is high
Latency (skid buffer)	1-2 clock cycles	Registered path from input to output
Backpressure response	1 clock cycle	TREADY deassertion is absorbed by the skid buffer
Maximum frequency	Design target only	Not characterized; no area or Fmax data published

1.11 Related Documentation

- **AXIS4 Modules** - AXI4-Stream components
- **AXI5 Modules** - AXI5 protocol components
- **APB5 Modules** - APB5 protocol components
- **GAXI Modules** - Generic AXI utilities (FIFOs, CDC)

1.12 References

1.12.1 Specifications

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051B, Apr. 2021 (defines AXI4-Stream and AXI5-Stream). developer.arm.com/documentation/ihi0051/b – §2.2 “Handshake signaling”, §2.3 “Wake-up signaling” (`TWAKEUP`), ch. 5 “Interface parity protection” (§5.3 odd parity and 8-bit granularity, Table 5-1 check signals), implemented as specified when `ENABLE_PARITY=1` (issue #102). Citation only.
- Arm. *AMBA 4 AXI4-Stream Protocol Specification*, Version 1.0, ARM IHI 0051A, Mar. 2010. developer.arm.com/documentation/ihi0051/a – the AXI4-Stream base (ch. 2 “Interface Signals”). Citation only.

1.12.2 Papers and textbooks

- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (AXI Stream bus protocol overview) – textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and valid-ready slices) – textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`)

1.12.3 Source Code

- RTL: `rtl/amba/axis5/`
- Tests: `val/amba/test_axis5*.py`
- Framework: `bin/TBClasses/components/axis4/`

Last Updated: 2026-07-19

1.13 Navigation

- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

2 AXIS5 Master

Module: `axis5_master.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

2.1 Overview

The AXIS5 Master implements an AXI5-Stream master interface with the AMBA5 extensions — wake-up signaling for power management — plus optional interface parity protection (the IHI 0051B ch. 5 check signals). Buffering comes from an internal skid buffer, which is what keeps the timing clean at the endpoint boundary.

2.1.1 Key Features

- AXI4-Stream data path (TDATA, TSTRB, TLAST, TID, TDEST, TUSER, TVALID/TREADY)
- TWAKEUP: Wake-up signaling for power management
- Interface parity (IHI 0051B ch. 5): TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK – odd parity, one check bit per 8 bits, gated by `ENABLE_PARITY`
- Internal skid buffer for backpressure handling
- Configurable data, ID, destination, and user signal widths
- Check-signal error detection (sticky `parity_error`)
- Busy status indication

Signal coverage: this module does not implement TKEEP, TPOISON, or any chunking sideband. See Implemented Signal Set for the full list of deviations from the ARM signal set.

2.1.2 AXIS5 Extensions Over AXIS4

Feature	AXIS4	AXIS5	ARM standard signal?
Wake-up signal	None	TWAKEUP (configurable)	Yes, AXI5-Stream addition
Interface parity	None	T*CHK check signals, odd parity, 1 bit per 8 bits (configurable)	Yes, AXI5-Stream ch. 5 (Check_Type = Odd_Parity_Byte_All)
Parity error detection	None	Built-in sticky	No - implementation status output

Feature	AXIS4	AXIS5	ARM standard signal?
		parity_error flag	

2.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable the IHI 0051B ch. 5 check signals (1 = Check_Type Odd_Parity_Byte_All, 0 = False)

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0, the spec's `Check_Type = False`: the `*chk` ports are still present but outputs are tied 0 and inputs are ignored.

2.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes

Name	Derivation	Meaning
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands
TDATACHK_W	SW	TDATACHK width, TDATA_WIDTH/8 (Table 5-1)
TSTRBCHK_W	ceil(SW/8)	TSTRBCHK width, ceil(TDATA_WIDTH/64)
TIDCHK_W / TDESTCHK_W / TUSERCHK_W	ceil(width/8)	TIDCHK / TDESTCHK / TUSERCHK widths

`TSize` (skid payload width) is a body `localparam` now, not a parameter.

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS clock
aresetn	1	Input	AXIS active-low asynchronous reset

2.3.2 FUB AXIS5 Interface (Input Side)

Port	Width	Direction	Description
fub_axis_tdata	DW	Input	Transfer data
fub_axis_tstrb	SW	Input	Transfer byte strobes
fub_axis_tlast	1	Input	Last transfer in packet
fub_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
fub_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
fub_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
fub_axis_tvalid	1	Input	Transfer valid
fub_axis_tready	1	Output	Transfer ready (skid buffer not full)
fub_axis_twakeup	1	Input	Wake-up signal (AXIS5 extension)
fub_axis_tdatachk	TDATACHK_W	Input	TDATACHK from the producer, carried with the beat
fub_axis_tstrbchk	TSTRBCHK_W	Input	TSTRBCHK, carried

Port	Width	Direction	Description
fub_axis_tlastchk	1	Input	TLASTCHK, carried
fub_axis_tidchk	TIDCHK_W	Input	TIDCHK, carried
fub_axis_tdestchk	TDESTCHK_W	Input	TDESTCHK, carried
fub_axis_tuserchk	TUSERCHK_W	Input	TUSERCHK, carried

2.3.3 Master AXIS5 Interface (Output Side)

Port	Width	Direction	Description
m_axis_tdata	DW	Output	Transfer data
m_axis_tstrb	SW	Output	Transfer byte strobes
m_axis_tlast	1	Output	Last transfer in packet
m_axis_tid	IW_WIDTH	Output	Transfer ID (optional)
m_axis_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
m_axis_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
m_axis_tvalid	1	Output	Transfer valid
m_axis_tready	1	Input	Transfer ready from downstream
m_axis_twakeup	1	Output	Wake-up signal (AXIS5 extension)
m_axis_tdatachk	TDATACHK_W	Output	TDATACHK: odd parity, bit n covers tdata[8n+7:8n]
m_axis_tstrbchk	TSTRBCHK_W	Output	TSTRBCHK: odd parity over tstrb, 1 bit per 8 strobes

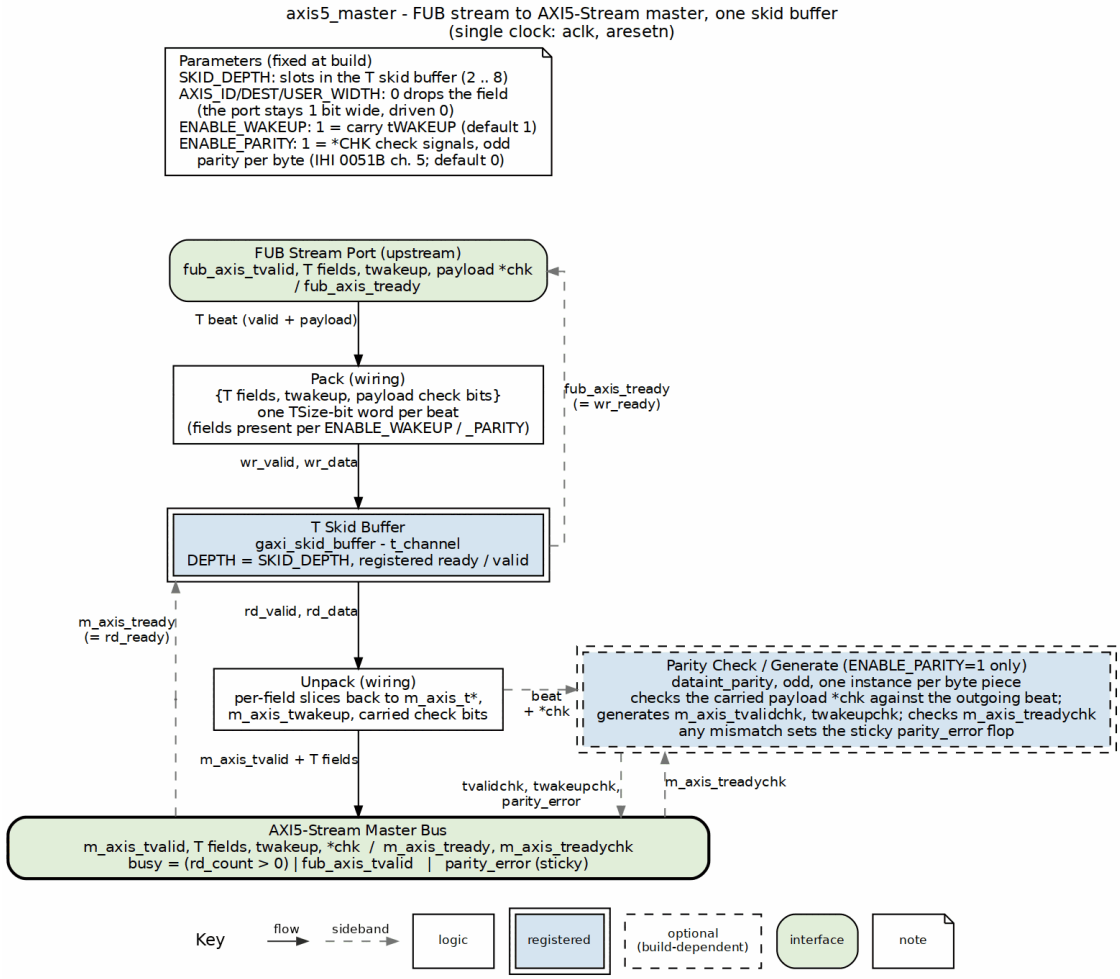
Port	Width	Direction	Description
m_axis_tlastchk	1	Output	TLASTCHK (= ~tlast for a correct beat)
m_axis_tidchk	TIDCHK_W	Output	TIDCHK
m_axis_tdestchk	TDESTCHK_W	Output	TDESTCHK
m_axis_tuserchk	TUSERCHK_W	Output	TUSERCHK
m_axis_tvalidchk	1	Output	TVALIDCHK = ~m_axis_tvalid, generated here every cycle
m_axis_treadychk	1	Input	TREADYCHK from the receiver, checked every cycle out of reset
m_axis_twakeupchk	1	Output	TWAKEUPCHK = ~m_axis_twakeup, generated here every cycle

2.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Sticky: a payload check mismatched on a TVALID cycle, or TREADYCHK was not ~TREADY

2.4 Functional Description

2.4.1 Block diagram



axis5_master block diagram

The `axis4_master` path with the AXI5-Stream additions riding the skid word: `twakeup` (`ENABLE_WAKEUP`) and, with `ENABLE_PARITY=1`, the payload check bits. The dashed parity block checks the carried check bits against the outgoing beat with `dataint_parity` (odd, one instance per byte piece), generates `m_axis_tvalidchk` and `m_axis_twakeupchk`, checks `m_axis_treadychk`, and sets the sticky `parity_error` on any mismatch.

2.4.2 Skid Buffer Operation

The module uses an internal `gaxi_skid_buffer` to: - Accept incoming transfers even when downstream is not ready - Prevent backpressure propagation - Provide registered outputs for timing closure - Track buffer occupancy via `busy` signal

2.4.3 Packet Packing/Unpacking

Conditional packing based on configuration:

```
// Full feature set (ENABLE_WAKEUP=1, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup, tdatachk, tstrbchk,
tlastchk, tidchk, tdestchk, tuserchk}

// Wake-up only (ENABLE_WAKEUP=1, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup}

// Parity only (ENABLE_WAKEUP=0, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, tdatachk, tstrbchk, tlastchk,
tidchk, tdestchk, tuserchk}

// Base AXIS4 (ENABLE_WAKEUP=0, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser}
```

2.4.4 Interface Parity (ENABLE_PARITY=1)

The check signals follow IHI 0051B ch. 5 with `Check_Type = Odd_Parity_Byte_All`:

- **Odd parity** (§5.3): a check bit plus the bits it covers always hold an odd number of ones.
- **Granularity** (§5.3, Table 5-1): check bit [n] covers bits [8n+7:8n] of its signal; when the signal is not a whole number of bytes, the top check bit covers the short remainder. TDATACHK is TDATA_WIDTH/8 bits, TSTRBCHK ceil(TDATA_WIDTH/64), TIDCHK/TDESTCHK/TUSERCHK ceil(width/8), and TLASTCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK one bit each (a one-bit check is the inversion of its signal).
- Every check bit is computed by one `rtl/common/dataint_parity` instance (`CHUNKS=1`, `parity_type=0` = odd) over its own piece, which gives the spec layout at any width.

What this module does with each signal:

Check signal	Here	Check Enable (Table 5-1)
TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK	Supplied by the FUB on `fub_axis_t*chk`, carried	TVALID

Check signal	Here	Check Enable (Table 5-1)
	through the skid buffer with the beat, driven on `m_axis_t*chk`, and checked on the output side	
TVALIDCHK	Generated: `m_axis_tvalidchk = ~m_axis_tvalid`	ARESETn (every cycle)
TWAKEUPCHK	Generated: `m_axis_twakeupchk = ~m_axis_twakeup`	ARESETn
TREADYCHK	Input from the receiver, checked every cycle	ARESETn

`parity_error` latches (until reset) on any payload mismatch in a cycle where `m_axis_tvalid` is high, or on any cycle where `m_axis_readychk` is not the inversion of `m_axis_tready`. When a

parameter removes TID, TDEST or TUSER, the spec omits its check (§5.3): the port stays, is driven with the odd parity of the tied-zero field, and is not checked.

Because the payload checks are carried, not regenerated, the output-side check validates the path *through this module* against the parity the producer computed. The receiving endpoint runs its own check (`axis5_slave` checks on its input). If the upstream FUB does not compute the checks, leave `ENABLE_PARITY=0`; tying `fub_axis_t*chk` to a constant while parity is enabled sets `parity_error` on the first beat whose parity disagrees.

2.4.5 Busy Signal

The `busy` output indicates: - Input side has valid data (`fub_axis_tvalid`) - Skid buffer contains data (`int_t_count > 0`)

2.5 Timing Characteristics

2.5.1 Basic Transfer with Wake-up

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axis_tvalid/tready
- fub_axis_tdata
- fub_axis_tlast
- fub_axis_twakeup (AXIS5 extension)
- m_axis_tvalid/tready
- m_axis_tdata
- m_axis_twakeup
- busy

2.5.2 Transfer with Parity Error

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- m_axis_tvalid / m_axis_tready
- m_axis_tdata, m_axis_tdatachk (carried)
- m_axis_treadychk (from the receiver)
- parity_error (sticky flag)

2.5.3 Skid Buffer Backpressure

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axis_tvalid/tready
- m_axis_tvalid/tready (downstream blocked)
- int_t_count (buffer fill level)
- busy

2.6 Usage Examples

2.6.1 Basic Configuration

```
axis5_master #(
    .SKID_DEPTH       (4),
    .AXIS_DATA_WIDTH  (64),
    .AXIS_ID_WIDTH    (8),
    .AXIS_DEST_WIDTH  (4),
    .AXIS_USER_WIDTH  (1),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (0)
) u_axis5_master (
    .aclk               (axis_clk),
    .aresetn            (axis_rst_n),

    // FUB interface (from upstream)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tdatachk  ('0),  // check inputs ignored when
ENABLE_PARITY=0
    .fub_axis_tstrbchk  ('0),
    .fub_axis_tlastchk  (1'b0),
    .fub_axis_tidchk    ('0),
    .fub_axis_tdestchk  ('0),
    .fub_axis_tuserchk  ('0),

    // Master AXIS5 interface (to downstream)
    .m_axis_tdata       (m_axis_tdata),
    .m_axis_tstrb       (m_axis_tstrb),
    .m_axis_tlast       (m_axis_tlast),
    .m_axis_tid         (m_axis_tid),
    .m_axis_tdest       (m_axis_tdest),
    .m_axis_tuser       (m_axis_tuser),
    .m_axis_tvalid      (m_axis_tvalid),
    .m_axis_tready      (m_axis_tready),
    .m_axis_twakeup     (m_axis_twakeup),
    .m_axis_tdatachk    (),  // tied 0 when ENABLE_PARITY=0
    .m_axis_tstrbchk    (),
    .m_axis_tlastchk    (),
    .m_axis_tidchk      (),
```

```
    .m_axis_tdestchk     (),
    .m_axis_tuserchk     (),
    .m_axis_tvalidchk    (),
    .m_axis_treadychk    (1'b0),
    .m_axis_twakeupchk   (),

    // Status
    .busy                (axis_busy),
    .parity_error        ()  // Not used when ENABLE_PARITY=0
);
```

2.6.2 With Parity Protection

```
axis5_master #(
    .AXIS_DATA_WIDTH  (32),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)  // IHI 0051B ch. 5 check signals
) u_axis5_master_parity (
    .aclk                (axis_clk),
    .aresetn             (axis_rst_n),

    // FUB interface with its payload check signals
    .fub_axis_tdata      (fub_tdata),
    .fub_axis_tstrb      (fub_tstrb),
    .fub_axis_tlast      (fub_tlast),
    .fub_axis_tid        (fub_tid),
    .fub_axis_tdest      (fub_tdest),
    .fub_axis_tuser      (fub_tuser),
    .fub_axis_tvalid     (fub_tvalid),
    .fub_axis_tready     (fub_tready),
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tdatachk   (fub_tdatachk),   // 4 bits for 32-bit data
    .fub_axis_tstrbchk   (fub_tstrbchk),   // 1 bit
    .fub_axis_tlastchk   (fub_tlastchk),
    .fub_axis_tidchk     (fub_tidchk),     // 1 bit for an 8-bit TID
    .fub_axis_tdestchk   (fub_tdestchk),
    .fub_axis_tuserchk   (fub_tuserchk),

    // Master AXIS5 interface
    .m_axis_tdata        (m_axis_tdata),
    .m_axis_tstrb        (m_axis_tstrb),
    .m_axis_tlast        (m_axis_tlast),
    .m_axis_tid          (m_axis_tid),
    .m_axis_tdest        (m_axis_tdest),
    .m_axis_tuser        (m_axis_tuser),
    .m_axis_tvalid       (m_axis_tvalid),
    .m_axis_tready       (m_axis_tready),
    .m_axis_twakeup      (m_axis_twakeup),
```

```
    .m_axis_tdatachk     (m_axis_tdatachk),
    .m_axis_tstrbchk     (m_axis_tstrbchk),
    .m_axis_tlastchk     (m_axis_tlastchk),
    .m_axis_tidchk       (m_axis_tidchk),
    .m_axis_tdestchk     (m_axis_tdestchk),
    .m_axis_tuserchk     (m_axis_tuserchk),
    .m_axis_tvalidchk    (m_axis_tvalidchk),
    .m_axis_treadychk    (m_axis_treadychk),
    .m_axis_twakeupchk   (m_axis_twakeupchk),

    // Status - monitor check-signal errors
    .busy                (axis_busy),
    .parity_error        (axis_parity_err)  // Sticky error flag
);

// Error handling
always_ff @(posedge axis_clk or negedge axis_rst_n) begin
    if (!axis_rst_n)
        error_count <= '0;
    else if (axis_parity_err && !prev_parity_err)
        error_count <= error_count + 1;
end
```

2.7 Design Notes

2.7.1 AXIS5 vs AXIS4 Differences

Feature	AXIS4	AXIS5
Wake-up signal	Not present	TWAKEUP (optional)
Interface parity	Not present	IHI 0051B ch. 5 T*CHK check signals (optional)
Power management	Limited	Enhanced via TWAKEUP
Data integrity	CRC/checksum in TUSER	Odd-parity check signals option
TKEEP	Not present	Not present

2.7.2 Skid Buffer Sizing

- **Typical depth:** 4-8 entries
- Deeper buffers:
  - Absorb longer downstream stalls
  - Increase area and latency
- Shallower buffers:
  - Lower latency
  - May cause upstream backpressure

Recommendation: depth 4 covers most applications; go deeper if your downstream stalls frequently.

2.7.3 Parity Implementation

When `ENABLE_PARITY=1`: - **Overhead:** 1 bit per 8 bits of every protected signal (a 512-bit TDATA adds 64 TDATACHK bits and 8 TSTRBCHK bits), plus 1 bit each for TLAST, TVALID, TREADY and TWAKEUP - **Detection:** Any odd number of bit errors within an 8-bit piece; two flipped bits in the same piece are undetected - **Correction:** None - error flag only - **Polarity:** Odd, as IHI 0051B §5.3 requires; there is no parameter to select even parity - **Use case:** Low-cost error detection in reliable environments

For stronger protection, use: - CRC in TUSER field - ECC (external module) - End-to-end checksums at packet level

2.7.4 Optional Signal Widths

Setting width parameters to 0 disables optional signals: - `AXIS_ID_WIDTH=0` → TID tied to 0, saves area - `AXIS_DEST_WIDTH=0` → TDEST tied to 0 - `AXIS_USER_WIDTH=0` → TUSER tied to 0

Internal logic uses `IW_WIDTH = (IW > 0) ? IW : 1` to avoid zero-width signals.

2.8 Related Modules

- **AXIS5 Slave** - AXIS5 slave interface
- **AXIS5 Master CG** - Clock-gated variant with power management
- **AXIS5 Slave CG** - Clock-gated slave variant
- **AXIS4 Master** - AXIS4 version for comparison
- **AMBA5 Overview** - AMBA5 specifications and extensions

2.9 Testing

`val/amba/test_axis5_master.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_master.py -v
```

2.10 References

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051B, Apr. 2021 (defines AXI4-Stream and AXI5-Stream). developer.arm.com/documentation/ihi0051/b — §2.2 “Handshake signaling”, §2.3 “Wake-up signaling” (`TWAKEUP`), ch. 5 “Interface parity protection” (§5.3 odd parity and the 8-bit granularity, Table 5-1 check-signal names, widths and Check Enable terms), implemented here as specified when `ENABLE_PARITY=1` (issue #102). Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (AXI Stream bus protocol overview) — textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and valid-ready slices) — textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

2.11 Navigation

- ← Back to AXIS5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

3 AXIS5 Master with Clock Gating

Module: `axis5_master_cg.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

3.1 Overview

The AXIS5 Master CG (Clock Gated) wraps the standard AXIS5 master with the AMBA clock gate controller, giving you an AXI5-Stream master interface that gates its own clock during idle periods. Full AXIS5 functionality survives the wrapping — wake-up signaling, the optional IHI 0051B ch. 5 check signals, the skid buffer, all of it.

3.1.1 Key Features

- Same AXI5-Stream data path as `axis5_master` (see its signal coverage note; TKEEP and TPOISON are not implemented)
- Automatic clock gating during idle periods for power savings
- TWAKEUP: Wake-up signaling integration with clock gate control
- Interface parity (IHI 0051B ch. 5): the T*CHK check signals, odd parity, one bit per 8 bits, gated by `ENABLE_PARITY` (see AXIS5 Master)
- Configurable idle count threshold for gating activation
- Internal skid buffer for backpressure handling
- Check-signal error detection (sticky `parity_error`)
- Clock gating status output

3.1.2 Power Management Features

Feature	Implementation	Benefit
Clock gating	Automatic via idle detection	Reduces dynamic power
Wake-up integration	TWAKEUP prevents gating	Preserves responsiveness
Configurable threshold	Programmable idle count	Tune power vs. latency
Activity detection	Multi-source OR logic	Comprehensive coverage

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable the IHI 0051B ch. 5 check signals (1 = Check_Type Odd_Parity_Byte_All, 0 = False)
CG_IDLE_COUNT_WIDTH	int	4	Clock gate idle counter width

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0, the spec's `Check_Type = False`: the `*chk` ports are still present but outputs are tied 0 and inputs are ignored.

3.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name

Name	Derivation	Meaning
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes
ICW	CG_IDLE_COUNT_WIDTH	Idle count width short name
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands
TDATACHK_W	SW	TDATACHK width, TDATA_WIDTH/8 (Table 5-1)
TSTRBCHK_W	ceil(SW/8)	TSTRBCHK width, ceil(TDATA_WIDTH/64)
TIDCHK_W / TDESTCHK_W / TUSERCHK_W	ceil(width/8)	TIDCHK / TDESTCHK / TUSERCHK widths

3.3 Ports

3.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS ungated clock (always running)
aresetn	1	Input	AXIS active-low asynchronous reset

3.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=allow gating, 0=force always on)
cfg_cg_idle_count	ICW	Input	Idle clock cycles before gating activates

3.3.3 FUB AXIS5 Interface (Input Side)

Port	Width	Direction	Description
fub_axis5_tdata	DW	Input	Transfer data
fub_axis5_tstrb	SW	Input	Transfer byte strobes
fub_axis5_tlast	1	Input	Last transfer in packet
fub_axis5_tid	IW_WIDTH	Input	Transfer ID (optional)
fub_axis5_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
fub_axis5_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
fub_axis5_t	1	Input	Transfer valid

Port	Width	Direction	Description
valid			
fub_axis5_tready	1	Output	Transfer ready (skid buffer not full)
fub_axis5_twakeup	1	Input	Wake-up signal (prevents clock gating)
fub_axis5_tdatachk	TDATACHK_W	Input	TDATACHK (odd parity, 1 bit per byte)
fub_axis5_tstrbchk	TSTRBCHK_W	Input	TSTRBCHK
fub_axis5_tlastchk	1	Input	TLASTCHK
fub_axis5_tidchk	TIDCHK_W	Input	TIDCHK
fub_axis5_tdestchk	TDESTCHK_W	Input	TDESTCHK
fub_axis5_tuserchk	TUSERCHK_W	Input	TUSERCHK

3.3.4 Master AXIS5 Interface (Output Side)

Port	Width	Direction	Description
m_axis5_tdata	DW	Output	Transfer data
m_axis5_tstrb	SW	Output	Transfer byte strobes
m_axis5_tlast	1	Output	Last transfer in packet
m_axis5_tid	IW_WIDTH	Output	Transfer ID (optional)
m_axis5_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
m_axis5_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
m_axis5_tv	1	Output	Transfer valid

Port	Width	Direction	Description
alid			
m_axis5_tready	1	Input	Transfer ready from downstream
m_axis5_twakeup	1	Output	Wake-up signal
m_axis5_tdatachk	TDATACHK_W	Output	TDATACHK (odd parity, 1 bit per byte)
m_axis5_tstrbchk	TSTRBCHK_W	Output	TSTRBCHK
m_axis5_tlastchk	1	Output	TLASTCHK
m_axis5_tidchk	TIDCHK_W	Output	TIDCHK
m_axis5_tdestchk	TDESTCHK_W	Output	TDESTCHK
m_axis5_tuserchk	TUSERCHK_W	Output	TUSERCHK
m_axis5_tvalidchk	1	Output	TVALIDCHK = ~m_axis5_tvalid
m_axis5_treadychk	1	Input	TREADYCHK from the receiver, checked every cycle out of reset
m_axis5_twakeupchk	1	Output	TWAKEUPCHK = ~m_axis5_twakeup

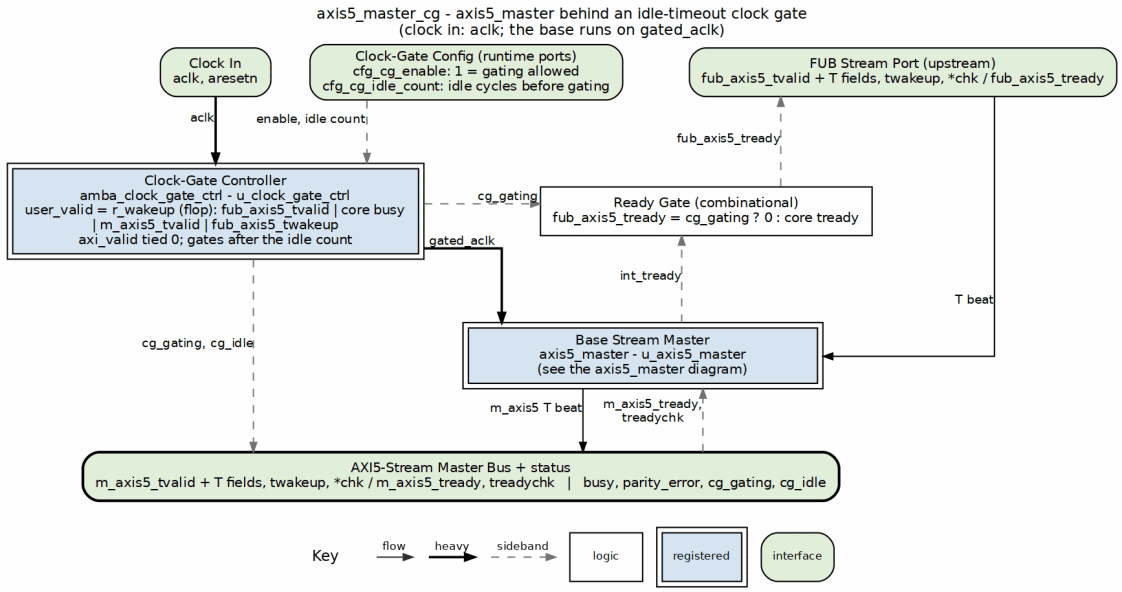
3.3.5 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Sticky check-signal error (see AXIS5 Master)
cg_gating	1	Output	Clock gating active status

Port	Width	Direction	Description
			(1=gated, 0=running)
cg_idle	Out	1	Activity terms quiet (registered ~wakeup).

3.4 Functional Description

3.4.1 Block diagram



axis5_master_cg block diagram

The base `axis5_master` is one box here (see its own diagram). The wrapper's `amba_clock_gate_ctrl` is kept awake by a registered activity term, `r_wakeup` = `fub_axis5_tvalid | core busy | m_axis5_tvalid | fub_axis5_twakeup`, and its gated clock (heavy) drives the base. A ready gate holds `fub_axis5_tready` low while the clock is gated.

3.4.2 Clock Gating Control

Wake-up detection logic:

```
r_wakeup <= fub_axis5_tvalid ||  // Input has data
            core_busy ||          // Core processing
            m_axis5_tvalid ||     // Output has data
            fub_axis5_twakeup;    // Explicit wake-up
```

The clock gate controller: 1. Monitors `r_wakeup` for activity 2. If idle (r_wakeup=0) for `cfg_cg_idle_count` cycles, gates the clock 3. On activity (r_wakeup=1), ungates the clock 4. Respects `cfg_cg_enable` (clock forced on when 0)

3.4.3 Wake-up and Gating Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. AXI5-Stream is a **two-stage** family: `r_wakeup` in this wrapper, then a second `r_wakeup` inside

amba_clock_gate_ctrl. The first gated-clock rising edge available to the core therefore arrives 3 aclk cycles after activity asserts. Budget accordingly:

Event	Latency	Path
Activity to first usable gated-clock edge	3 aclk cycles (2 register stages)	fub_axis5_tvalid to r_wakeup to controller r_wakeup to combinational ICG enable to next edge
Last activity to clock gated	cfg_cg_idle_count + 3 aclk cycles	cfg_cg_idle_count + 1 after the internal wakeup deasserts, plus two wake-up register stages

The wake-up path is **not** zero-latency. Nothing is lost during those cycles: fub_axis5_tready is masked with !cg_gating, so it reads 0 while the clock is stopped. The register behind it holds its last value when the clock stops – it does NOT fall on its own – so without that mask a producer would see a still-asserted READY and lose a beat (TASK-076, fixed 2026-09-02) and the producer simply holds TVALID until the clock resumes. Both registers and the idle counter run on the ungated aclk, so the wake-up path stays live even while the core clock is stopped.

3.4.4 Reset Behavior

Reset is safe with respect to gating; the clock is guaranteed to be running whenever reset is asserted:

- aresetn is asynchronous and is passed **ungated** to both the clock gate controller and the core, so its assertion edge does not depend on the gated clock.
- r_wakeup in this wrapper and r_wakeup inside amba_clock_gate_ctrl both reset to 1, which forces the ICG enable active. The clock therefore runs during reset and for at least the idle countdown after reset deassertion.
- clock_gate_ctrl loads its idle counter with cfg_cg_idle_count on reset, so gating cannot re-engage until a full idle period has elapsed after release.

Because reset removal is asynchronous, apply the usual practice of releasing aresetn synchronously to aclk at the system level.

3.4.5 Power Saving Mechanism

Clock gating states:

Condition	r_wakeup	Clock State	Power State
Active transfer	1	Running	Full power
Buffer has data	1	Running	Full power
Wake-up asserted	1	Running	Full power
Idle < threshold	0	Running	Full power
Idle ≥ threshold	0	Gated	Low power

Benefits: - Reduces dynamic power during idle periods - Wake-up on new transfers costs 2 register stages; the first usable gated-clock edge arrives 3 `aclk` cycles after activity (see Wake-up and Gating Latency) - No protocol impact (transparent to upstream/downstream)

3.4.6 Integration with AXIS5 Extensions

Wake-up signal dual purpose: 1. **Protocol:** Power management hint to downstream 2. **Clock gating:** Prevents local clock gating

This dual use is what guarantees wake-up events are never missed because the clock happened to be gated.

3.4.7 Idle Count Configuration

Typical values for `cfg_cg_idle_count`: - **Aggressive power saving:** 2-4 cycles - Quick gating, may have frequent gate/ungate transitions - Best for bursty traffic with long idle periods - **Balanced:** 8-16 cycles - Reduces gate/ungate overhead - Good for moderate traffic patterns - **Conservative:** 32-64 cycles - Only gates during extended idle - Minimal gate/ungate overhead

Trade-off: lower count means more power savings but more gate transitions, and each transition has its own dynamic overhead.

3.5 Timing Characteristics

3.5.1 Clock Gating Activation

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk (always running)
- r_wakeup (activity detection)
- cfg_cg_idle_count (threshold)
- gated_clk (starts gating after threshold)
- cg_gating (status)
- fub_axis5_tvalid (new transfer wakes up)

3.5.2 Wake-up Signal Preventing Gating

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axis5_twakeup (asserted)
- r_wakeup (stays high)
- gated_clk (remains running)
- cg_gating (stays 0)

3.5.3 Transfer During Idle Period

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- gated_clk (gated, then ungates on transfer)
- fub_axis5_tvalid (new transfer)
- r_wakeup (0 → 1 transition)
- cg_gating (1 → 0)
- m_axis5_tvalid (output after ungate)

3.6 Usage Examples

3.6.1 Basic Clock-Gated Configuration

```
axis5_master_cg #(
    .SKID_DEPTH             (4),
    .AXIS_DATA_WIDTH        (64),
    .AXIS_ID_WIDTH          (8),
    .AXIS_DEST_WIDTH        (4),
    .AXIS_USER_WIDTH        (1),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (0),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (axis_clk),
    .aresetn                (axis_rst_n),

    // Clock gating configuration
    .cfg_cg_enable            (1'b1),        // Enable clock gating
    .cfg_cg_idle_count        (4'd8),        // Gate after 8 idle
cycles

    // FUB interface (from upstream)
    .fub_axis5_tdata        (fub_tdata),
    .fub_axis5_tstrb        (fub_tstrb),
    .fub_axis5_tlast        (fub_tlast),
    .fub_axis5_tid          (fub_tid),
    .fub_axis5_tdest        (fub_tdest),
    .fub_axis5_tuser        (fub_tuser),
    .fub_axis5_tvalid       (fub_tvalid),
    .fub_axis5_tready       (fub_tready),
    .fub_axis5_twakeup      (fub_twakeup),
    .fub_axis5_tdatachk      ('0),
    .fub_axis5_tstrbchk      ('0),
    .fub_axis5_tlastchk      (1'b0),
    .fub_axis5_tidchk        ('0),
    .fub_axis5_tdestchk      ('0),
    .fub_axis5_tuserchk      ('0),

    // Master AXIS5 interface (to downstream)
    .m_axis5_tdata          (m_axis_tdata),
    .m_axis5_tstrb          (m_axis_tstrb),
    .m_axis5_tlast          (m_axis_tlast),
    .m_axis5_tid            (m_axis_tid),
    .m_axis5_tdest          (m_axis_tdest),
    .m_axis5_tuser          (m_axis_tuser),
    .m_axis5_tvalid         (m_axis_tvalid),
    .m_axis5_tready         (m_axis_tready),
```

```
    .m_axis5_twakeup        (m_axis_twakeup),
    .m_axis5_tdatachk        (),
    .m_axis5_tstrbchk        (),
    .m_axis5_tlastchk        (),
    .m_axis5_tidchk          (),
    .m_axis5_tdestchk        (),
    .m_axis5_tuserchk        (),
    .m_axis5_tvalidchk      (),
    .m_axis5_treadychk      (1'b0),
    .m_axis5_twakeupchk     (),

    // Status
    .busy                   (axis_busy),
    .parity_error           (),
    .cg_gating      (axis_clk_gated)  // Monitor gating status
);
```

3.6.2 Dynamic Clock Gating Control

```
// Runtime control of clock gating
logic cg_enable;
logic [3:0] cg_idle_count;

axis5_master_cg #(
    .AXIS_DATA_WIDTH        (32),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (sys_clk),
    .aresetn                (sys_rst_n),

    // Dynamic configuration via registers
    .cfg_cg_enable            (cg_enable),
    .cfg_cg_idle_count        (cg_idle_count),

    // ... ports ...

    .busy                   (axis_busy),
    .parity_error           (parity_err),
    .cg_gating      (clk_gated_status)
);

// Configuration register interface
always_ff @(posedge cfg_clk or negedge cfg_rst_n) begin
    if (!cfg_rst_n) begin
        cg_enable <= 1'b1;      // Default: gating enabled
```

```
        cg_idle_count <= 4'd16; // Default: 16 cycle threshold
    end else if (cfg_write) begin
        case (cfg_addr)
            ADDR_CG_CTRL: cg_enable <= cfg_wdata[0];
            ADDR_CG_IDLE: cg_idle_count <= cfg_wdata[3:0];
        endcase
    end
end

// Monitor power savings
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)
        gated_cycle_count <= '0;
    else if (clk_gated_status)
        gated_cycle_count <= gated_cycle_count + 1;
end
```

3.6.3 Power Profiling Example

```
// Instantiate both gated and non-gated versions for comparison
axis5_master_cg #(
    .AXIS_DATA_WIDTH    (64),
    .CG_IDLE_COUNT_WIDTH(4)
) u_gated (
    .aclk               (clk),
    .aresetn            (rst_n),
    .cfg_cg_enable        (1'b1),
    .cfg_cg_idle_count    (4'd8),
    // ... ports ...
    .cg_gating  (gated_status)
);

// Track gating statistics
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (gated_status)
            gated_cycles <= gated_cycles + 1;
    end
end

// Calculate power savings percentage
assign gated_duty_pct = (gated_cycles * 100) / total_cycles;
```

Read this metric carefully. `gated_duty_pct` is the fraction of time the clock was gated, not a power saving. Actual dynamic power saved depends on the switching activity that would have occurred during those cycles; leakage is unaffected by gating; and the gating logic and its clock-tree branch consume power themselves. Use the number as a gating-effectiveness indicator, and get real figures from a power analysis tool fed a VCD or SAIF from a representative workload.

3.7 Design Notes

3.7.1 Clock Gating vs. Non-Gated Variant

Aspect	axis5_master	axis5_master_cg
Area	Smaller	+5-10% (clock gate logic)
Dynamic power	Higher	Lower (gating reduces)
Static power	Same	Same
Latency	Lower	Same in steady state; +3 `aclk` cycles when waking from a gated state (two register stages)
Complexity	Lower	Higher (gating control)
Use case	Always-on systems	Power-sensitive designs

When to use clock-gated variant: - Battery-powered devices - Bursty traffic patterns with idle periods - Power budget constraints - SoC integration with power domains

3.7.2 Wake-up Integration Benefits

TWAKEUP integration with clock gating provides: 1. **Event preservation:** Wake-up events prevent clock gating 2. **Bounded restart:** Clock activation takes a fixed 2 `aclk` cycles, and no transfer is dropped while it happens 3. **Protocol compliance:** Wake-up propagated correctly 4. **Power efficiency:** Only wake when necessary

3.7.3 Gate Transition Overhead

Each clock gate transition has overhead: - **Gate activation:** `cfg_cg_idle_count` + 1 cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count` + 3 cycles after the last bus activity - **Ungate activation:** two wake-up register stages; first usable gated-clock edge 3 cycles after activity asserts - **Dynamic power:** Gate/ungate switching consumes power

Recommendation: set `cfg_cg_idle_count` high enough to amortize the transition overhead.

3.7.4 Clock Domain Considerations

- **Input signals:** Must be on `aclk` domain (always running)
- **Core logic:** Operates on `gated_clk` domain
- **Output signals:** Driven by `gated_clk` domain
- **Status outputs:** `cg_gating` on `aclk` domain

No CDC synchronizers are needed, because `gated_clk` is `aclk` passed through an ICG cell: same frequency, same phase, no independent clock source. There is still physical work to do:

- **Clock tree synthesis:** `gated_clk` must be balanced against `aclk`. The wake-up registers and `cg_gating` sit on `aclk` while the core sits on `gated_clk`, so uncontrolled skew between the two branches eats directly into the timing budget on every path that crosses between them.
- **ICG placement:** place the ICG cell close to the root of the gated branch so the insertion delay it adds is common to the whole core.
- **FPGA targets:** `clock_gate_ctrl` instantiates an ICG cell, which is an ASIC library primitive. On FPGAs use a clock-enable style implementation or a device buffer with an enable (for example `BUFGCE`) instead; a bare ICG will not map cleanly.

3.8 Related Modules

- **AXIS5 Master** - Base AXIS5 master without clock gating
- **AXIS5 Slave CG** - Clock-gated slave variant
- **AXIS5 Slave** - Base AXIS5 slave
- **AMBA Clock Gate Controller** - Clock gating controller
- **AMBA5 Power Management** - AMBA5 power features

3.9 Testing

`val/amba/test_axis5_master_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_master_cg.py -v
```

3.10 References

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051B, Apr. 2021 (defines AXI4-Stream and AXI5-Stream). developer.arm.com/documentation/ihi0051/b – ch. 2 “Interface Signals” including §2.3 “Wake-up signaling” (`TWAKEUP`), and ch. 5 “Interface parity protection” (§5.3 odd parity, Table 5-1 check signals), implemented by the wrapped block when `ENABLE_PARITY=1` (issue #102): the protocol of the wrapped AXI5-Stream block. Citation only.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

3.11 Navigation

- ← Back to AXIS5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

4 AXIS5 Slave

Module: `axis5_slave.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

4.1 Overview

The AXIS5 Slave implements an AXI5-Stream slave interface with the AMBA5 extensions — wake-up signaling for power management — plus optional interface parity protection (the IHI 0051B ch. 5 check signals). It accepts incoming stream data and buffers it through an internal skid buffer, which is what keeps backpressure off your upstream link.

4.1.1 Key Features

- AXI4-Stream data path (TDATA, TSTRB, TLAST, TID, TDEST, TUSER, TVALID/TREADY)
- TWAKEUP: Wake-up signaling for power management
- Interface parity (IHI 0051B ch. 5): TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK – odd parity, one check bit per 8 bits, gated by `ENABLE_PARITY`
- Internal skid buffer for backpressure handling
- Configurable data, ID, destination, and user signal widths
- Check-signal error detection (sticky `parity_error`)
- Busy status indication

Signal coverage: this module does not implement TKEEP, TPOISON, or any chunking sideband. See Implemented Signal Set for the full list of deviations from the ARM signal set.

4.1.2 AXIS5 Extensions Over AXIS4

Feature	AXIS4	AXIS5	ARM standard signal?
Wake-up signal	None	TWAKEUP (configurable)	Yes, AXI5-Stream addition
Interface parity	None	T*CHK check signals, odd parity, 1 bit per 8 bits (configurable)	Yes, AXI5-Stream ch. 5 (Check_Type = Odd_Parity_Byte_All)
Parity error detection	None	Built-in sticky	No - implementation status output

Feature	AXIS4	AXIS5	ARM standard signal?
		parity_error flag	

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable the IHI 0051B ch. 5 check signals (1 = Check_Type Odd_Parity_Byte_All, 0 = False)

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0, the spec's `Check_Type = False`: the `*chk` ports are still present but outputs are tied 0 and inputs are ignored.

4.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes

Name	Derivation	Meaning
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands
TDATACHK_W	SW	TDATACHK width, TDATA_WIDTH/8 (Table 5-1)
TSTRBCHK_W	ceil(SW/8)	TSTRBCHK width, ceil(TDATA_WIDTH/64)
TIDCHK_W / TDESTCHK_W / TUSERCHK_W	ceil(width/8)	TIDCHK / TDESTCHK / TUSERCHK widths

`TSize` (skid payload width) is a body `localparam` now, not a parameter.

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS clock
aresetn	1	Input	AXIS active-low asynchronous reset

4.3.2 Slave AXIS5 Interface (Input Side)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Transfer data from upstream
s_axis_tstrb	SW	Input	Transfer byte strobes
s_axis_tlast	1	Input	Last transfer in packet
s_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
s_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
s_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
s_axis_tvalid	1	Input	Transfer valid from upstream
s_axis_tready	1	Output	Transfer ready (skid buffer not full)
s_axis_twakeup	1	Input	Wake-up signal (AXIS5 extension)
s_axis_tdatachk	TDATACHK_W	Input	TDATACHK: odd parity, bit n covers tdata[8n+7:8n]
s_axis_tstrbchk	TSTRBCHK_W	Input	TSTRBCHK: odd parity over tstrb, 1 bit per 8 strobes
s_axis_tlastchk	1	Input	TLASTCHK

Port	Width	Direction	Description
s_axis_tidchk	TIDCHK_W	Input	TIDCHK
s_axis_tdestchk	TDESTCHK_W	Input	TDESTCHK
s_axis_tuserchk	TUSERCHK_W	Input	TUSERCHK
s_axis_tvalidchk	1	Input	TVALIDCHK, checked every cycle out of reset
s_axis_treadychk	1	Output	TREADYCHK = ~s_axis_tready, generated here every cycle
s_axis_twakeupchk	1	Input	TWAKEUPCHK, checked every cycle when ENABLE_WAKEUP=1

4.3.3 FUB AXIS5 Interface (Output Side to Backend)

Port	Width	Direction	Description
fub_axis_tdata	DW	Output	Transfer data to backend
fub_axis_tstrb	SW	Output	Transfer byte strobes
fub_axis_tlast	1	Output	Last transfer in packet
fub_axis_tid	IW_WIDTH	Output	Transfer ID (optional)
fub_axis_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
fub_axis_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
fub_axis_tvalid	1	Output	Transfer valid to backend
fub_axis_tready	1	Input	Transfer ready from backend
fub_axis_t	1	Output	Wake-up signal (AXIS5

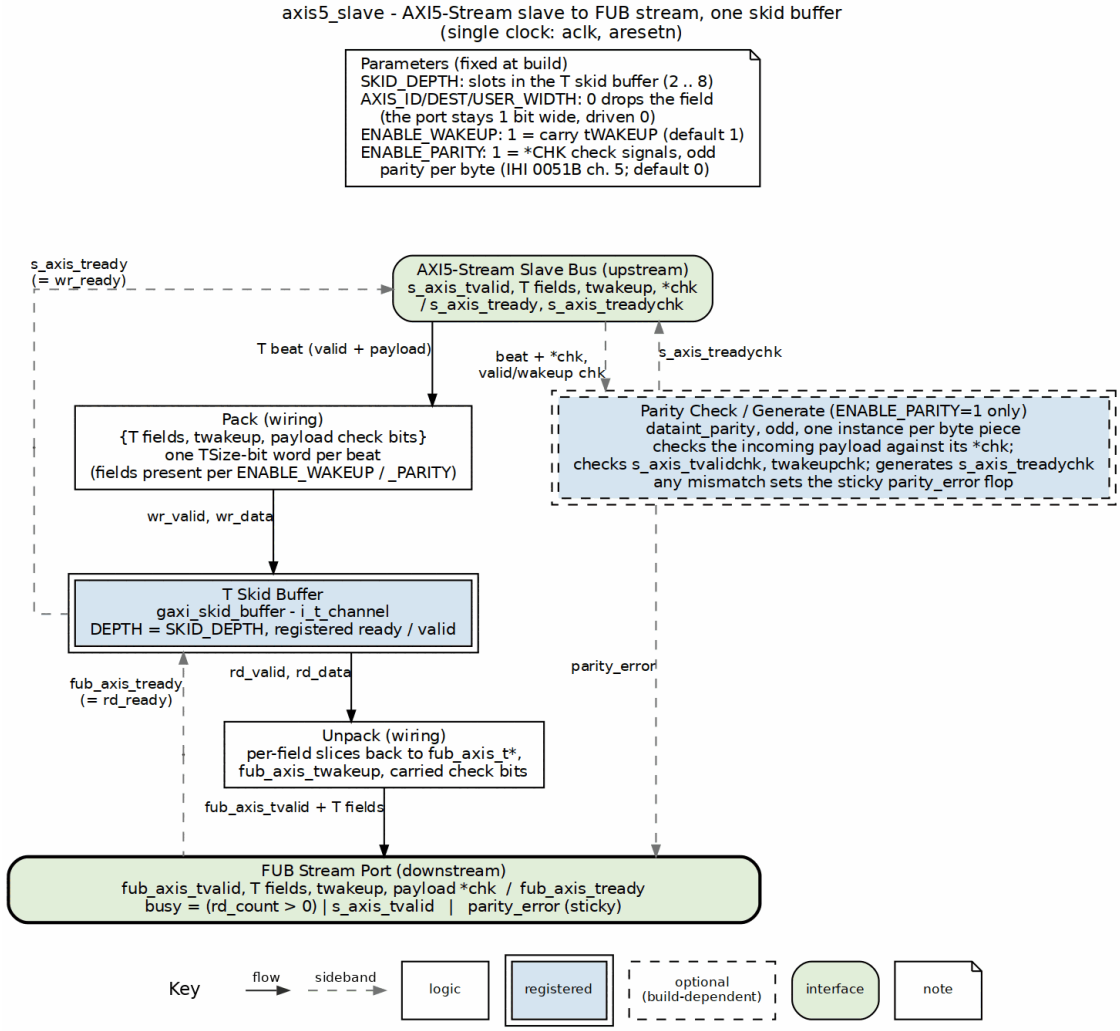
Port	Width	Direction	Description
wakeup			extension)
fub_axis_tdatachk	TDATACHK_W	Output	TDATACHK, carried with the beat
fub_axis_tstrbchk	TSTRBCHK_W	Output	TSTRBCHK, carried
fub_axis_tlastchk	1	Output	TLASTCHK, carried
fub_axis_tidchk	TIDCHK_W	Output	TIDCHK, carried
fub_axis_tdestchk	TDESTCHK_W	Output	TDESTCHK, carried
fub_axis_tuserchk	TUSERCHK_W	Output	TUSERCHK, carried

4.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Sticky: a payload check mismatched on an s_axis TVALID cycle, or TVALIDCHK/TWAKEUPCHK was not the inversion of its signal

4.4 Functional Description

4.4.1 Block diagram



axis5_slave block diagram

The mirror of `axis5_master`. With `ENABLE_PARITY=1` the dashed parity block checks the incoming beat against its check bits on the bus side, checks `s_axis_tvalidchk` and `s_axis_twakeupchk`, generates `s_axis_treadychk`, and sets the sticky `parity_error`; the check bits and `twakeup` then ride the skid word to the FUB side.

4.4.2 Skid Buffer Operation

The module uses an internal `gaxi_skid_buffer` to: - Accept incoming transfers even when backend is not ready - Prevent backpressure to upstream - Provide registered outputs for timing closure - Track buffer occupancy via `busy` signal

4.4.3 Packet Packing/Unpacking

Conditional packing based on configuration:

```
// Full feature set (ENABLE_WAKEUP=1, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup, tdatachk, tstrbchk,
tlastchk, tidchk, tdestchk, tuserchk}

// Wake-up only (ENABLE_WAKEUP=1, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup}

// Parity only (ENABLE_WAKEUP=0, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, tdatachk, tstrbchk, tlastchk,
tidchk, tdestchk, tuserchk}

// Base AXIS4 (ENABLE_WAKEUP=0, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser}
```

4.4.4 Interface Parity (ENABLE_PARITY=1)

The check signals follow IHI 0051B ch. 5 with `Check_Type = Odd_Parity_Byte_All`:

- **Odd parity** (§5.3): a check bit plus the bits it covers always hold an odd number of ones.
- **Granularity** (§5.3, Table 5-1): check bit [n] covers bits [8n+7:8n] of its signal; when the signal is not a whole number of bytes, the top check bit covers the short remainder. TDATACHK is TDATA_WIDTH/8 bits, TSTRBCHK ceil(TDATA_WIDTH/64), TIDCHK/TDESTCHK/TUSERCHK ceil(width/8), and TLASTCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK one bit each (a one-bit check is the inversion of its signal).
- Every check bit is computed by one `rtl/common/dataint_parity` instance (`CHUNKS=1`, `parity_type=0` = odd) over its own piece, which gives the spec layout at any width.

What this module does with each signal:

Check signal	Here	Check Enable (Table 5-1)
TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK	Checked on `s_axis`, then carried through	TVALID

Check signal	Here	Check Enable (Table 5-1)
	the skid buffer to `fub_axis_t*chk`	
TVALIDCHK	Input, checked against `~s_axis_tvalid`	ARESETn (every cycle)
TWAKEUPCHK	Input, checked against `~s_axis_twakeup` (only when `ENABLE_WAKEUP=1`)	ARESETn
TREADYCHK	Generated: `s_axis_treadychk = ~s_axis_tready`	ARESETn

`parity_error` latches (until reset) on any payload mismatch in a cycle where `s_axis_tvalid` is high, whether or not the beat is accepted, or on any cycle where TVALIDCHK or TWAKEUPCHK is wrong. When a parameter removes TID, TDEST or TUSER, the spec omits its check (§5.3): the input is ignored and the `fub_axis` check output is driven as the odd parity of the tied-zero field.

Checking at the input covers the upstream link and catches errors as early as possible. This is the receiver-side check of the pair; `axis5_master` checks on its output side, which covers its own internal data path.

4.4.5 Busy Signal

The `busy` output indicates: - Input side has valid data (`s_axis_tvalid`) - Skid buffer contains data (`int_t_count > 0`)

4.5 Timing Characteristics

4.5.1 Basic Receive with Wake-up

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tvalid/tready
- s_axis_tdata
- s_axis_tlast
- s_axis_twakeup (AXIS5 extension)
- fub_axis_tvalid/tready
- fub_axis_tdata
- fub_axis_twakeup
- busy

4.5.2 Receive with Parity Error

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tdata
- s_axis_tvalid, s_axis_tvalidchk
- s_axis_tdata, s_axis_tdatachk (received)
- parity_error (sticky flag set)

4.5.3 Skid Buffer Backpressure from Backend

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tvalid/tready
- fub_axis_tvalid/tready (backend blocked)
- int_t_count (buffer fill level)
- busy

4.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axis5_slave #(
    .SKID_DEPTH           (4),
    .AXIS_DATA_WIDTH      (32),
    .AXIS_ID_WIDTH        (8),
    .AXIS_DEST_WIDTH      (4),
    .AXIS_USER_WIDTH      (1),
    .ENABLE_WAKEUP        (1),
    .ENABLE_PARITY        (0)
) u_axis5_slave (
    .aclk                 (aclk),
    .aresetn              (aresetn),
    .s_axis_tdata         (s_axis_tdata),
    .s_axis_tstrb         (s_axis_tstrb),
    .s_axis_tlast         (s_axis_tlast),
    .s_axis_tid           (s_axis_tid),
    .s_axis_tdest         (s_axis_tdest),
    .s_axis_tuser         (s_axis_tuser),
    .s_axis_tvalid        (s_axis_tvalid),
    .s_axis_tready        (s_axis_tready),
    .s_axis_twakeup       (s_axis_twakeup),
    .s_axis_tdatachk      (s_axis_tdatachk),
    .s_axis_tstrbchk      (s_axis_tstrbchk),
    .s_axis_tlastchk      (s_axis_tlastchk),
    .s_axis_tidchk        (s_axis_tidchk),
    .s_axis_tdestchk      (s_axis_tdestchk),
    .s_axis_tuserchk      (s_axis_tuserchk),
    .s_axis_tvalidchk     (s_axis_tvalidchk),
    .s_axis_treadychk     (s_axis_treadychk),
    .s_axis_twakeupchk    (s_axis_twakeupchk),
    .fub_axis_tdata       (fub_axis_tdata),
    .fub_axis_tstrb       (fub_axis_tstrb),
    .fub_axis_tlast       (fub_axis_tlast),
    .fub_axis_tid         (fub_axis_tid),
    .fub_axis_tdest       (fub_axis_tdest),
    .fub_axis_tuser       (fub_axis_tuser),
    .fub_axis_tvalid      (fub_axis_tvalid),
    .fub_axis_tready      (fub_axis_tready),
    .fub_axis_twakeup     (fub_axis_twakeup),
    .fub_axis_tdatachk    (fub_axis_tdatachk),
    .fub_axis_tstrbchk    (fub_axis_tstrbchk),
    .fub_axis_tlastchk    (fub_axis_tlastchk),
    .fub_axis_tidchk      (fub_axis_tidchk),
    .fub_axis_tdestchk    (fub_axis_tdestchk),
    .fub_axis_tuserchk    (fub_axis_tuserchk),
```

```
    .busy                   (busy),
    .parity_error           (parity_error)
);
```

4.7 Design Notes

4.7.1 AXIS5 vs AXIS4 Differences

Feature	AXIS4	AXIS5
Wake-up signal	Not supported	TWAKEUP (optional)
Interface parity	Not supported	IHI 0051B ch. 5 T*CHK check signals (optional)
Power management	Limited	Enhanced via TWAKEUP
Data integrity	CRC/checksum in TUSER	Odd-parity check signals option

4.7.2 Parity Check Location

Key difference from AXIS5 Master: - **Slave:** checks on **input** (`s_axis_t*` vs `s_axis_t*chk`, plus TVALIDCHK/TWAKEUPCHK), and generates TREADYCHK - **Master:** checks on **output** (`m_axis_t*` vs the carried `m_axis_t*chk`, plus TREADYCHK), and generates TVALIDCHK/TWAKEUPCHK

Checking at the input means transmission errors get caught at the receiving side, as early as possible.

4.7.3 Skid Buffer Sizing

- **Typical depth:** 4-8 entries
- Deeper buffers:
  - Absorb longer backend stalls
  - Increase area and latency
- Shallower buffers:
  - Lower latency
  - May cause upstream backpressure

Recommendation: depth 4 covers most applications; go deeper if your backend stalls frequently.

4.7.4 Parity Implementation

When `ENABLE_PARITY=1`: - **Overhead:** 1 bit per 8 bits of every protected signal (a 512-bit TDATA adds 64 TDATACHK bits and 8 TSTRBCHK bits), plus 1 bit each for TLAST, TVALID, TREADY and TWAKEUP - **Detection:** Any odd number of bit errors within an 8-bit piece; two flipped bits in the same piece are undetected - **Correction:** None - error flag only - **Polarity:** Odd, as IHI 0051B §5.3 requires; there is no parameter to select even parity - **Use case:** Low-cost error detection in reliable environments

For stronger protection, use: - CRC in TUSER field - ECC (external module) - End-to-end checksums at packet level

4.7.5 Error Handling Strategy

The `parity_error` flag is **sticky** (latches high until reset): - Simplifies error detection (level-sensitive interrupt) - Software must clear by resetting module - For edge-sensitive interrupts, external logic must detect rising edge

Alternative: modify the module to add a `parity_error_clear` input if your system needs one.

4.8 Related Modules

- **AXIS5 Master** - AXIS5 master interface (transmit side)
- **AXIS5 Slave CG** - Clock-gated variant with power management
- **AXIS5 Master CG** - Clock-gated master variant
- **AXIS4 Slave** - AXIS4 version for comparison
- **AMBA5 Overview** - AMBA5 specifications and extensions

4.9 Testing

`val/amba/test_axis5_slave.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_slave.py -v
```

4.10 References

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051B, Apr. 2021 (defines AXI4-Stream and AXI5-Stream). developer.arm.com/documentation/ihi0051/b — §2.2 “Handshake signaling”, §2.3 “Wake-up signaling” (`TWAKEUP`), ch. 5 “Interface parity protection” (§5.3 odd parity and the 8-bit granularity, Table 5-1 check-signal names, widths and Check Enable terms), implemented here as specified when `ENABLE_PARITY=1` (issue #102). Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (AXI Stream bus protocol overview) — textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and valid-ready slices) — textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

4.11 Navigation

- ← Back to AXIS5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

5 AXIS5 Slave with Clock Gating

Module: `axis5_slave_cg.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

5.1 Overview

The AXIS5 Slave CG (Clock Gated) wraps the standard AXIS5 slave with the AMBA clock gate controller, giving you an AXI5-Stream slave interface that gates its own clock during idle periods. Full AXIS5 functionality survives the wrapping — wake-up signaling, the optional IHI 0051B ch. 5 check signals, the skid buffer, all of it.

5.1.1 Key Features

- Same AXI5-Stream data path as `axis5_slave` (see its signal coverage note; TKEEP and TPOISON are not implemented)
- Automatic clock gating during idle periods for power savings
- TWAKEUP: Wake-up signaling integration with clock gate control
- Interface parity (IHI 0051B ch. 5): the T*CHK check signals, odd parity, one bit per 8 bits, gated by `ENABLE_PARITY` (see AXIS5 Slave)
- Configurable idle count threshold for gating activation
- Internal skid buffer for backpressure handling
- Check-signal error detection on the input side (sticky `parity_error`)
- Clock gating status output

5.1.2 Power Management Features

Feature	Implementation	Benefit
Clock gating	Automatic via idle detection	Reduces dynamic power
Wake-up integration	TWAKEUP prevents gating	Preserves responsiveness
Configurable threshold	Programmable idle count	Tune power vs. latency
Activity detection	Multi-source OR logic	Comprehensive coverage

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable the IHI 0051B ch. 5 check signals (1 = Check_Type Odd_Parity_Byte_All, 0 = False)
CG_IDLE_COUNT_WIDTH	int	4	Clock gate idle counter width

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0, the spec's `Check_Type = False`: the `*chk` ports are still present but outputs are tied 0 and inputs are ignored.

5.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name

Name	Derivation	Meaning
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes
ICW	CG_IDLE_COUNT_WIDTH	Idle count width short name
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands
TDATACHK_W	SW	TDATACHK width, TDATA_WIDTH/8 (Table 5-1)
TSTRBCHK_W	ceil(SW/8)	TSTRBCHK width, ceil(TDATA_WIDTH/64)
TIDCHK_W / TDESTCHK_W / TUSERCHK_W	ceil(width/8)	TIDCHK / TDESTCHK / TUSERCHK widths

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS ungated clock (always running)
aresetn	1	Input	AXIS active-low asynchronous reset

5.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=allow gating, 0=force always on)
cfg_cg_idle_count	ICW	Input	Idle clock cycles before gating activates

5.3.3 Slave AXIS5 Interface (Input Side)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Transfer data from upstream
s_axis_tstrb	SW	Input	Transfer byte strobes
s_axis_tlast	1	Input	Last transfer in packet
s_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
s_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
s_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
s_axis_tvalid	1	Input	Transfer valid from upstream
s_axis_trea	1	Output	Transfer ready (skid

Port	Width	Direction	Description
dy			buffer not full)
s_axis_twakeup	1	Input	Wake-up signal (prevents clock gating)
s_axis_tdatachk	TDATACHK_W	Input	TDATACHK (odd parity, 1 bit per byte)
s_axis_tstrbchk	TSTRBCHK_W	Input	TSTRBCHK
s_axis_tlastchk	1	Input	TLASTCHK
s_axis_tidchk	TIDCHK_W	Input	TIDCHK
s_axis_tdestchk	TDESTCHK_W	Input	TDESTCHK
s_axis_tuserchk	TUSERCHK_W	Input	TUSERCHK
s_axis_tvalidchk	1	Input	TVALIDCHK, checked every cycle out of reset (while the clock runs)
s_axis_treadychk	1	Output	TREADYCHK = ~s_axis_tready – generated in this wrapper from the MASKED ready, so it reads 1 while gated
s_axis_twakeupchk	1	Input	TWAKEUPCHK, checked when ENABLE_WAKEUP=1

5.3.4 FUB AXIS5 Interface (Output Side to Backend)

Port	Width	Direction	Description
fub_axis5_tdata	DW	Output	Transfer data to backend
fub_axis5_tstrb	SW	Output	Transfer byte strobes
fub_axis5_t	1	Output	Last transfer in packet

Port	Width	Direction	Description
last			
fub_axis5_tid	IW_WIDTH	Output	Transfer ID (optional)
fub_axis5_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
fub_axis5_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
fub_axis5_tvalid	1	Output	Transfer valid to backend
fub_axis5_tready	1	Input	Transfer ready from backend
fub_axis5_twakeup	1	Output	Wake-up signal
fub_axis5_tdatachk	TDATACHK_W	Output	TDATACHK (odd parity, 1 bit per byte)
fub_axis5_tstrbchk	TSTRBCHK_W	Output	TSTRBCHK
fub_axis5_tlastchk	1	Output	TLASTCHK
fub_axis5_tidchk	TIDCHK_W	Output	TIDCHK
fub_axis5_tdestchk	TDESTCHK_W	Output	TDESTCHK
fub_axis5_tuserchk	TUSERCHK_W	Output	TUSERCHK

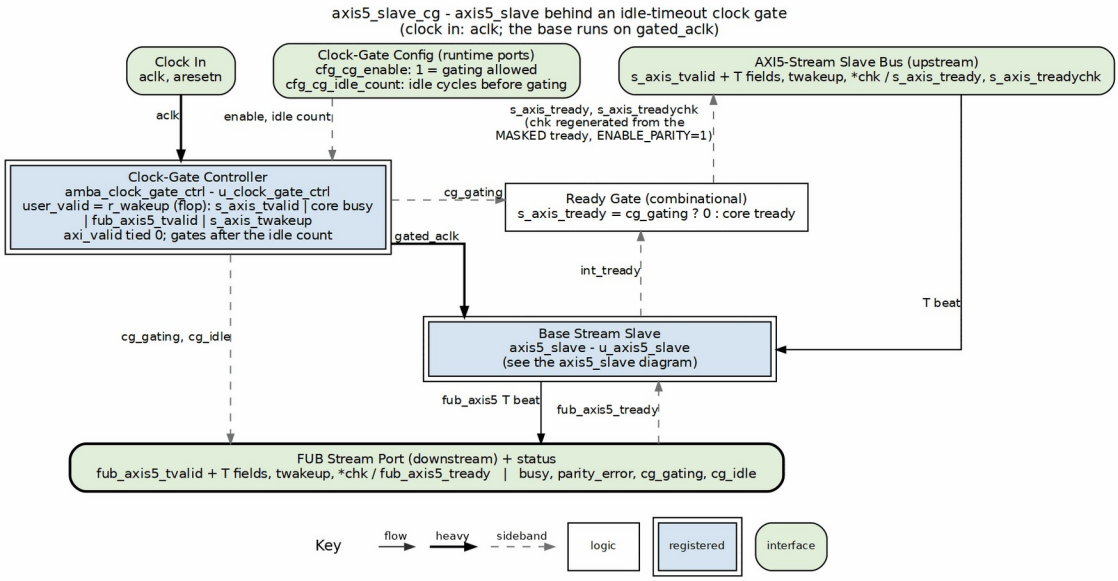
5.3.5 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Sticky check-signal error on the input side (see AXIS5 Slave)

Port	Width	Direction	Description
cg_gating	1	Output	Clock gating active status (1=gated, 0=running)
cg_idle	Out	1	Activity terms quiet (registered ~wakeup).

5.4 Functional Description

5.4.1 Block diagram



axis5_slave_cg block diagram

The base `axis5_slave` is one box here (see its own diagram). The wrapper's `amba_clock_gate_ctrl` is kept awake by the registered `r_wakeup` = `s_axis_tvalid | core busy | fub_axis5_tvalid | s_axis_twakeup`. A ready gate holds `s_axis_tready` low while the clock is gated, and `s_axis_treadychk` is regenerated from that masked ready so the check covers what the transmitter actually sees.

5.4.2 Clock Gating Control

Wake-up detection logic:

```
r_wakeup <= s_axis_tvalid ||    // Input has data
            core_busy ||         // Core processing
            fub_axis5_tvalid ||  // Backend has data
            s_axis_twakeup;      // Explicit wake-up
```

The clock gate controller: 1. Monitors `r_wakeup` for activity 2. If idle (r_wakeup=0) for `cfg_cg_idle_count` cycles, gates the clock 3. On activity (r_wakeup=1), ungates the clock 4. Respects `cfg_cg_enable` (clock forced on when 0)

5.4.3 Wake-up and Gating Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. AXI5-Stream is a **two-stage** family: `r_wakeup` in this wrapper, then a second `r_wakeup` inside

amba_clock_gate_ctrl. The first gated-clock rising edge available to the core therefore arrives 3 aclk cycles after activity asserts. Budget accordingly:

Event	Latency	Path
Activity to first usable gated-clock edge	3 aclk cycles (2 register stages)	s_axis_tvalid to r_wakeup to controller r_wakeup to combinational ICG enable to next edge
Last activity to clock gated	cfg_cg_idle_count + 3 aclk cycles	cfg_cg_idle_count + 1 after the internal wakeup deasserts, plus two wake-up register stages

The wake-up path is **not** zero-latency. Nothing is lost during those cycles: s_axis_tready is masked with !cg_gating, so it reads 0 while the clock is stopped. The register behind it holds its last value when the clock stops – it does NOT fall on its own – so without that mask a producer would see a still-asserted READY and lose a beat (TASK-076, fixed 2026-09-02) and the upstream master simply holds TVALID until the clock resumes. Both registers and the idle counter run on the ungated aclk, so the wake-up path stays live even while the core clock is stopped.

5.4.4 Reset Behavior

Reset is safe with respect to gating; the clock is guaranteed to be running whenever reset is asserted:

- aresetn is asynchronous and is passed **ungated** to both the clock gate controller and the core, so its assertion edge does not depend on the gated clock.
- r_wakeup in this wrapper and r_wakeup inside amba_clock_gate_ctrl both reset to 1, which forces the ICG enable active. The clock therefore runs during reset and for at least the idle countdown after reset deassertion.
- clock_gate_ctrl loads its idle counter with cfg_cg_idle_count on reset, so gating cannot re-engage until a full idle period has elapsed after release.

Because reset removal is asynchronous, apply the usual practice of releasing aresetn synchronously to aclk at the system level.

5.4.5 Power Saving Mechanism

Clock gating states:

Condition	r_wakeup	Clock State	Power State
Receiving data	1	Running	Full power
Buffer has data	1	Running	Full power
Backend processing	1	Running	Full power
Wake-up asserted	1	Running	Full power
Idle < threshold	0	Running	Full power
Idle ≥ threshold	0	Gated	Low power

Benefits: - Reduces dynamic power during idle periods - Wake-up on new transfers costs 2 register stages; the first usable gated-clock edge arrives 3 `aclk` cycles after activity (see Wake-up and Gating Latency) - No protocol impact (transparent to upstream/backend)

5.4.6 Check-signal Error Detection

When `ENABLE_PARITY=1`: - The payload checks (`s_axis_t*chk`) and TVALIDCHK/TWAKEUPCHK are checked on the **input side** by the wrapped `axis5_slave` - `parity_error` flag is sticky (latches until reset) - No transfer can slip past the check while gated: `s_axis_tready` is masked to 0 while gated, so no transfer is accepted until the clock has resumed - **TREADYCHK covers the masked ready.** The core's READY register holds its last value when the clock stops, which is why the wrapper masks `s_axis_tready`; the core's own TREADYCHK would then disagree with the pin. The wrapper therefore leaves the core's `s_axis_treadychk` unconnected and regenerates it from the masked `s_axis_tready` (one `dataint_parity`, odd), so the pair is consistent in every cycle, gated or not

5.4.7 Idle Count Configuration

Typical values for `cfg_cg_idle_count`: - **Aggressive power saving:** 2-4 cycles - Quick gating, may have frequent gate/ungate transitions - Best for bursty traffic with long idle periods - **Balanced:** 8-16 cycles - Reduces gate/ungate overhead - Good for moderate traffic patterns - **Conservative:** 32-64 cycles - Only gates during extended idle - Minimal gate/ungate overhead

Trade-off: lower count means more power savings but more gate transitions, and each transition has its own dynamic overhead.

5.5 Timing Characteristics

5.5.1 Clock Gating Activation on Slave

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk (always running)
- s_axis_tvalid (input activity)
- r_wakeup (activity detection)
- cfg_cg_idle_count (threshold)
- gated_clk (starts gating after threshold)
- cg_gating (status)
- New s_axis_tvalid arrival ungates clock

5.5.2 Wake-up Preventing Gating During Receive

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_twakeup (asserted during transfer)
- s_axis_tvalid
- r_wakeup (stays high)
- gated_clk (remains running)
- cg_gating (stays 0)
- fub_axis5_tvalid (output forwarded)

5.5.3 Backend Backpressure with Clock Gating

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tvalid (upstream sending)
- fub_axis5_tready (backend blocked)
- core_busy (stays high)
- r_wakeup (stays high)
- gated_clk (remains running due to busy)
- cg_gating (stays 0 during backpressure)

5.6 Usage Examples

5.6.1 Basic Clock-Gated Slave

```
axis5_slave_cg #(
    .SKID_DEPTH             (4),
    .AXIS_DATA_WIDTH        (64),
    .AXIS_ID_WIDTH          (8),
    .AXIS_DEST_WIDTH        (4),
    .AXIS_USER_WIDTH        (1),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (0),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_slave_cg (
    .aclk                   (axis_clk),
    .aresetn                (axis_rst_n),

    // Clock gating configuration
    .cfg_cg_enable            (1'b1),        // Enable clock gating
    .cfg_cg_idle_count        (4'd8),        // Gate after 8 idle
cycles

    // Slave interface (from upstream)
    .s_axis_tdata           (s_axis_tdata),
    .s_axis_tstrb           (s_axis_tstrb),
    .s_axis_tlast           (s_axis_tlast),
    .s_axis_tid             (s_axis_tid),
    .s_axis_tdest           (s_axis_tdest),
    .s_axis_tuser           (s_axis_tuser),
    .s_axis_tvalid          (s_axis_tvalid),
    .s_axis_tready          (s_axis_tready),
    .s_axis_twakeup         (s_axis_twakeup),
    .s_axis_tdatachk         ('0),
    .s_axis_tstrbchk         ('0),
    .s_axis_tlastchk         (1'b0),
    .s_axis_tidchk           ('0),
    .s_axis_tdestchk         ('0),
    .s_axis_tuserchk         ('0),
    .s_axis_tvalidchk       (1'b0),
    .s_axis_treadychk       (),
    .s_axis_twakeupchk      (1'b0),

    // FUB interface (to backend)
    .fub_axis5_tdata        (fub_tdata),
    .fub_axis5_tstrb        (fub_tstrb),
    .fub_axis5_tlast        (fub_tlast),
    .fub_axis5_tid          (fub_tid),
    .fub_axis5_tdest        (fub_tdest),
```

```
    .fub_axis5_tuser        (fub_tuser),
    .fub_axis5_tvalid       (fub_tvalid),
    .fub_axis5_tready       (fub_tready),
    .fub_axis5_twakeup      (fub_twakeup),
    .fub_axis5_tdatachk      (),
    .fub_axis5_tstrbchk      (),
    .fub_axis5_tlastchk      (),
    .fub_axis5_tidchk        (),
    .fub_axis5_tdestchk      (),
    .fub_axis5_tuserchk      (),

    // Status
    .busy                   (axis_busy),
    .parity_error           (),
    .cg_gating      (axis_clk_gated)  // Monitor gating status
);
```

5.6.2 With Parity Protection and Power Monitoring

```
// Clock-gated slave with parity and comprehensive monitoring
axis5_slave_cg #(
    .AXIS_DATA_WIDTH        (32),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),  // IHI 0051B ch. 5 check signals
    .CG_IDLE_COUNT_WIDTH    (4)
) u_rx_slave_cg (
    .aclk                   (sys_clk),
    .aresetn                (sys_rst_n),

    // Dynamic clock gating control
    .cfg_cg_enable            (cfg_cg_enable),
    .cfg_cg_idle_count        (cfg_cg_threshold),

    // Slave interface with its check signals
    .s_axis_tdata           (rx_tdata),
    .s_axis_tstrb           (rx_tstrb),
    .s_axis_tlast           (rx_tlast),
    .s_axis_tid             (rx_tid),
    .s_axis_tdest           (rx_tdest),
    .s_axis_tuser           (rx_tuser),
    .s_axis_tvalid          (rx_tvalid),
    .s_axis_tready          (rx_tready),
    .s_axis_twakeup         (rx_twakeup),
    .s_axis_tdatachk         (rx_tdatachk),
    .s_axis_tstrbchk         (rx_tstrbchk),
    .s_axis_tlastchk         (rx_tlastchk),
    .s_axis_tidchk           (rx_tidchk),
    .s_axis_tdestchk         (rx_tdestchk),
```

```
    .s_axis_tuserchk         (rx_tuserchk),
    .s_axis_tvalidchk       (rx_tvalidchk),
    .s_axis_treadychk       (rx_treadychk),
    .s_axis_twakeupchk      (rx_twakeupchk),

    // FUB interface
    .fub_axis5_tdata        (proc_tdata),
    .fub_axis5_tstrb        (proc_tstrb),
    .fub_axis5_tlast        (proc_tlast),
    .fub_axis5_tid          (proc_tid),
    .fub_axis5_tdest        (proc_tdest),
    .fub_axis5_tuser        (proc_tuser),
    .fub_axis5_tvalid       (proc_tvalid),
    .fub_axis5_tready       (proc_tready),
    .fub_axis5_twakeup      (proc_twakeup),
    .fub_axis5_tdatachk      (proc_tdatachk),
    .fub_axis5_tstrbchk      (proc_tstrbchk),
    .fub_axis5_tlastchk      (proc_tlastchk),
    .fub_axis5_tidchk        (proc_tidchk),
    .fub_axis5_tdestchk      (proc_tdestchk),
    .fub_axis5_tuserchk      (proc_tuserchk),

    // Status monitoring
    .busy                   (rx_busy),
    .parity_error           (rx_parity_err),
    .cg_gating      (rx_clk_gated)
);

// Error and power monitoring
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        parity_error_count <= '0;
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        // Count cycles
        total_cycles <= total_cycles + 1;
        if (rx_clk_gated)
            gated_cycles <= gated_cycles + 1;

        // Count parity errors
        if (rx_parity_err && !prev_parity_err)
            parity_error_count <= parity_error_count + 1;
        prev_parity_err <= rx_parity_err;
    end
end
```

```
// Calculate power savings
assign gated_duty_pct = (gated_cycles * 100) / total_cycles;
```

Read this metric carefully. `gated_duty_pct` is the fraction of time the clock was gated, not a power saving. Actual dynamic power saved depends on the switching activity that would have occurred during those cycles; leakage is unaffected by gating; and the gating logic and its clock-tree branch consume power themselves. Use the number as a gating-effectiveness indicator, and get real figures from a power analysis tool fed a VCD or SAIF from a representative workload.

5.6.3 Receive Pipeline with Power Management

```
// Network receive path with clock gating
axis5_slave_cg #(
    .AXIS_DATA_WIDTH        (512),  // Wide data path
    .AXIS_ID_WIDTH          (4),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),
    .CG_IDLE_COUNT_WIDTH    (6)     // Wider counter for longer idle
) u_rx_interface (
    .aclk                   (net_clk),
    .aresetn                (net_rst_n),

    // Aggressive clock gating for power
    .cfg_cg_enable            (1'b1),
    .cfg_cg_idle_count        (6'd4),  // Gate quickly after 4 idle
cycles

    // From network PHY
    .s_axis_tdata           (phy_rx_tdata),
    .s_axis_tstrb           (phy_rx_tstrb),
    .s_axis_tlast           (phy_rx_tlast),
    .s_axis_tid             (phy_rx_tid),
    .s_axis_tdest           (phy_rx_tdest),
    .s_axis_tuser           (phy_rx_tuser),
    .s_axis_tvalid          (phy_rx_tvalid),
    .s_axis_tready          (phy_rx_tready),
    .s_axis_twakeup         (phy_rx_twakeup),
    .s_axis_tdatachk         (phy_rx_tdatachk),
    .s_axis_tstrbchk         (phy_rx_tstrbchk),
    .s_axis_tlastchk         (phy_rx_tlastchk),
    .s_axis_tidchk           (phy_rx_tidchk),
    .s_axis_tdestchk         (phy_rx_tdestchk),
    .s_axis_tuserchk         (phy_rx_tuserchk),
    .s_axis_tvalidchk       (phy_rx_tvalidchk),
    .s_axis_treadychk       (phy_rx_treadychk),
    .s_axis_twakeupchk      (phy_rx_twakeupchk),

    // To packet processor
```

```
    .fub_axis5_tdata        (pkt_proc_tdata),
    .fub_axis5_tstrb        (pkt_proc_tstrb),
    .fub_axis5_tlast        (pkt_proc_tlast),
    .fub_axis5_tid          (pkt_proc_tid),
    .fub_axis5_tdest        (pkt_proc_tdest),
    .fub_axis5_tuser        (pkt_proc_tuser),
    .fub_axis5_tvalid       (pkt_proc_tvalid),
    .fub_axis5_tready       (pkt_proc_tready),
    .fub_axis5_twakeup      (pkt_proc_twakeup),
    .fub_axis5_tdatachk      (pkt_proc_tdatachk),
    .fub_axis5_tstrbchk      (pkt_proc_tstrbchk),
    .fub_axis5_tlastchk      (pkt_proc_tlastchk),
    .fub_axis5_tidchk        (pkt_proc_tidchk),
    .fub_axis5_tdestchk      (pkt_proc_tdestchk),
    .fub_axis5_tuserchk      (pkt_proc_tuserchk),

    // Status for system monitoring
    .busy                   (rx_busy),
    .parity_error           (rx_parity_err),
    .cg_gating      (rx_clk_gated)
);

// Interrupt on parity error
assign parity_err_irq = rx_parity_err && !prev_rx_parity_err;
```

5.7 Design Notes

5.7.1 Clock Gating vs. Non-Gated Variant

Aspect	axis5_slave	axis5_slave_cg
Area	Smaller	+5-10% (clock gate logic)
Dynamic power	Higher	Lower (gating reduces)
Static power	Same	Same
Latency	Lower	Same in steady state; +3 `aclk` cycles when waking from a gated state (two register stages)
Complexity	Lower	Higher (gating control)
Use case	Always-on systems	Power-sensitive designs

When to use clock-gated variant: - Battery-powered devices - Receive paths with bursty traffic - Power budget constraints - SoC integration with power domains

5.7.2 Slave-Specific Gating Considerations

Differences from master variant: 1. **Wake-up source:** Upstream signal (`s_axis_twakeup`) 2. **Activity detection:** Input valid (`s_axis_tvalid`) 3. **Backpressure:** Backend ready affects gating

Receive path characteristics: - Often idle waiting for upstream data - Good candidate for aggressive clock gating - Wake-up from upstream provides early notification

5.7.3 Gate Transition Overhead

Each clock gate transition has overhead: - **Gate activation:** `cfg_cg_idle_count` + 1 cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count` + 3 cycles after the last bus activity - **Ungate activation:** two wake-up register stages; first usable gated-clock edge 3 cycles after activity asserts - **Dynamic power:** Gate/ungate switching consumes power

Recommendation: set `cfg_cg_idle_count` high enough to amortize the transition overhead, especially in scenarios where gating engages frequently.

5.7.4 Parity Error Handling with Clock Gating

Parity errors detected on input side: - The comparison is combinational on `s_axis_t*` and `s_axis_t*chk`, so it is evaluated before data enters the skid buffer - The `parity_error` register is clocked by `gated_clk` and captures on any `s_axis_tvalid` cycle with a bad payload check (Check Enable = TVALID), or any cycle with a bad TVALIDCHK/TWAKEUPCHK. While the clock is stopped nothing is sampled, but no transfer can be accepted either: `s_axis_tready` is masked low until the clock resumes, and `s_axis_tvalid` wakes the clock first - Once set, the flag holds through subsequent gating; a stopped clock cannot clear a register

Important: the `parity_error` flag requires reset to clear — plan for error recovery.

5.7.5 Clock Domain Considerations

- **Input signals:** Sampled by the wake-up logic on `aclk` (always running) and by the core on `gated_clk`
- **Core logic:** Operates on `gated_clk` domain
- **Output signals:** Driven by `gated_clk` domain
- **Status outputs:** `cg_gating` on `aclk` domain

No CDC synchronizers are needed, because `gated_clk` is `aclk` passed through an ICG cell: same frequency, same phase, no independent clock source. There is still physical work to do:

- **Clock tree synthesis:** `gated_clk` must be balanced against `aclk`. The wake-up registers and `cg_gating` sit on `aclk` while the core sits on `gated_clk`, so uncontrolled skew between the two branches eats directly into the timing budget on every path that crosses between them.
- **ICG placement:** place the ICG cell close to the root of the gated branch so the insertion delay it adds is common to the whole core.
- **FPGA targets:** `clock_gate_ctrl` instantiates an ICG cell, which is an ASIC library primitive. On FPGAs use a clock-enable style implementation or a device buffer with an enable (for example `BUFGCE`) instead; a bare ICG will not map cleanly.

5.8 Related Modules

- **AXIS5 Slave** - Base AXIS5 slave without clock gating
- **AXIS5 Master CG** - Clock-gated master variant
- **AXIS5 Master** - Base AXIS5 master
- **AMBA Clock Gate Controller** - Clock gating controller
- **AMBA5 Power Management** - AMBA5 power features

5.9 Testing

`val/amba/test_axis5_slave_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_slave_cg.py -v
```

5.10 References

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051B, Apr. 2021 (defines AXI4-Stream and AXI5-Stream). developer.arm.com/documentation/ihi0051/b – ch. 2 “Interface Signals” including §2.3 “Wake-up signaling” (`TWAKEUP`), and ch. 5 “Interface parity protection” (§5.3 odd parity, Table 5-1 check signals), implemented by the wrapped block when `ENABLE_PARITY=1` (issue #102): the protocol of the wrapped AXI5-Stream block. Citation only.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

5.11 Navigation

- ← Back to AXIS5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index